

Towards Optimal Differential Attacks on FLY and PIPO

Insung Kim¹, Seonggyeom Kim², Sunyeop Kim¹,
Donggeun Kwon¹, Hanbeom Shin¹, Dongjae Lee³, Deukjo Hong⁴,
Jaechul Sung⁵, Seokhie Hong^{1*}

¹School of Cybersecurity, Korea University, Seoul, 02855, Republic of Korea.

²System LSI Business, Samsung Electronics, Hwaseong, 18448, Republic of Korea.

³Department of Convergence Security, Kangwon National University, Chuncheon, 24341, Gangwon State, Republic of Korea.

⁴Smart Grid Research Center, Department of Computer Sciences and Artificial Intelligence, Jeonbuk National University, Jeonju, 54896, Jeonbuk State, Republic of Korea.

⁵Department of Mathematics, University of Seoul, Seoul, 02504, Republic of Korea.

*Corresponding author(s). E-mail(s): shhong@korea.ac.kr;
Contributing authors: cmcom35@korea.ac.kr; jeffgyeom@gmail.com;
kin3548@gmail.com; donggeun.kwon@gmail.com;
newonetiger@korea.ac.kr; dongjae.lee@kangwon.ac.kr;
deukjo.hong@jbnu.ac.kr; jcsung@uos.ac.kr;

Abstract

Lightweight block ciphers such as PIPO and FLY are designed to operate efficiently and securely in constrained environments. While the differential attack on PIPO-64-128 has already been studied by the designers, no concrete differential attack had been conducted for PIPO-64-256 and FLY. Motivated by this gap, we revisit the security of PIPO against differential attacks and generalize the analysis framework to make it applicable to structurally related ciphers. Based on this generalized framework, we search for key-recovery-attack-friendly distinguishers and apply clustering techniques to enhance their effectiveness in key-recovery attacks. As a result, we improve the previously proposed differential attack on

PIPO-64-128, reducing the time complexity by a factor of $2^{31.7}$. Furthermore, we propose a 13-round differential attack on PIPO-64-256, which covers two more rounds than the previous result. We also apply the same methodology to FLY and present the first differential attack on 12-round FLY, reaching one round beyond the best-known distinguisher. We believe this work improves the understanding of the structures of FLY and PIPO, and provides a basis for future research on advanced key-recovery attacks for related cipher designs.

Keywords: Block ciphers, Differential cryptanalysis, Key-recovery attack, PIPO, FLY

1 Introduction

The design of lightweight cryptography has emerged as a major research topic in symmetric-key cryptography. This trend is driven by the growing demand for security in resource-constrained environments such as IoT devices and embedded systems. The early lightweight block cipher, PRESENT [1], focused on minimizing hardware circuit size and laid the foundation for efficient cryptographic design. Since then, lightweight block cipher designs have continued to evolve to achieve a balance between hardware efficiency and security across various applications.

The design of lightweight block ciphers that resist side-channel attacks and support efficient masking has emerged as an important topic in symmetric cryptography in recent years. Starting with Zorro [2] and further developed through the LS-design [3] approach focusing on masking efficiency, this research direction has produced ciphers such as Rectangle [4], FLY [5], and PIPO [6]. In designing such ciphers, a tight security analysis is essential to minimize the number of rounds.

Differential cryptanalysis [7] is one of the most powerful techniques for evaluating the security of block ciphers. Resistance against differential attacks is one of the main factors in determining the required number of rounds for a cipher. Inaccurate estimation of resistance against differential attacks can compromise the security of the cipher. For instance, SPEEDY [8] was broken by differential attacks due to insufficient security analysis and an inadequate number of rounds [9]. Therefore, accurate security evaluation is essential during the design phase to ensure an appropriate security margin.

Conventional differential cryptanalysis focuses on identifying a single differential characteristic with the maximum expected probability and applying it in key-recovery attacks. However, as demonstrated in [10], such characteristics do not always result in the most efficient key-recovery attacks.

Regarding PIPO, the designers provided extensive details on its resistance to differential cryptanalysis. However, to date, no third-party differential attack results have been reported in the single-key setting. In contrast, the designers of FLY did not present any analysis regarding differential distinguishers or key-recovery attacks. However, a differential distinguisher on FLY, based on the clustering effect, was proposed in [11], although it was not extended to a key-recovery attack.

1.1 Our Contributions

In this paper, we focus on the security of PIPO and FLY against differential attacks. To search for optimal differential distinguishers, we adopt the trail search methodology for AES-like ciphers proposed in [12] and apply a branch-and-bound algorithm to find all single differential trails with high probabilities. Our main contributions are summarized as follows:

- We revisit the differential attack framework originally proposed in [6] for PIPO and present a generalized version that naturally applies not only to PIPO-64-128 and PIPO-64-256, but also to other ciphers with similar structures, such as FLY.
- For PIPO, we apply a branch-and-bound algorithm to find all single differential trails with probabilities of at least 2^{-63} . Based on the 78,693 trails identified, we construct optimal key-recovery attacks on PIPO-64-128, achieving a complexity improvement of $2^{31.7}$ over the previous best result. Furthermore, we apply clustering techniques to enhance the distinguishing probabilities of the optimal trails.
- We extend this analysis to PIPO-64-256, establishing a 13-round key-recovery attack, two rounds beyond the previously analyzed rounds.
- We also apply the same methodology to the block cipher FLY, conducting the first key-recovery-focused analysis of its optimal single differential trails. Based on the results, we construct key-recovery attacks and further enhance the distinguishing probabilities of the relevant trails using clustering techniques. This enables a 12-round key-recovery attack, surpassing the reach of the best-known distinguishers by one round.

Table 1 summarizes the key-recovery results for PIPO-64-128, PIPO-64-256, and FLY.

1.2 Paper Outline

Section 2 provides the specifications of PIPO and FLY, along with the necessary preliminaries for understanding this paper. Section 3 revisits the key-recovery attack on PIPO presented in [6], and analyzes its time, memory, and data complexities. Section 4 introduces our methodology for searching key-recovery-attack friendly distinguishers. Sections 5 and 6 present our improved key-recovery attacks on PIPO and FLY, respectively. Section 7 concludes the paper. Our codes and more detailed results can be found in <https://github.com/kim-insung-wq/differential-attack>.

2 Preliminaries

In this section, we provide the relevant definitions and notations used in this paper. We detail the specifications of the R-Layer-based lightweight block ciphers, namely PIPO [6] and FLY [5], and explain the differential attack [7].

Table 1: A summary of the results

Cipher	Rounds	Source	Attack type	Time	Data	Memory
PIPO-64-128	9	[6]	Differential	$2^{124.90}$	$2^{57.41}$	2^{116}
	9	Sec. 5.1	Differential	$2^{93.85}$	$2^{62.42}$	2^{92}
PIPO-64-256	11 ^o	[6]	Differential	N/A	N/A	N/A
	13	Sec. 5.2	Differential	$2^{221.85}$	2^{64}	2^{220}
	13	Sec. 5.2.2	Differential	$2^{222.40}$	2^{63}	2^{220}
FLY	11*	[11]	Differential	$2^{99.23}$	$2^{58.77}$	2^{98}
	12	Sec. 6.2	Differential	$2^{126.045}$	2^{64}	2^{92}
	12	Sec. 6.3	Differential	$2^{127.405}$	$2^{61.13}$	2^{92}

^o The authors only claimed a differential attack up to 11 rounds without providing a concrete key-recovery complexity.

* Although the authors did not propose a key-recovery attack, we estimated the key-recovery complexity based on their distinguisher for comparison.

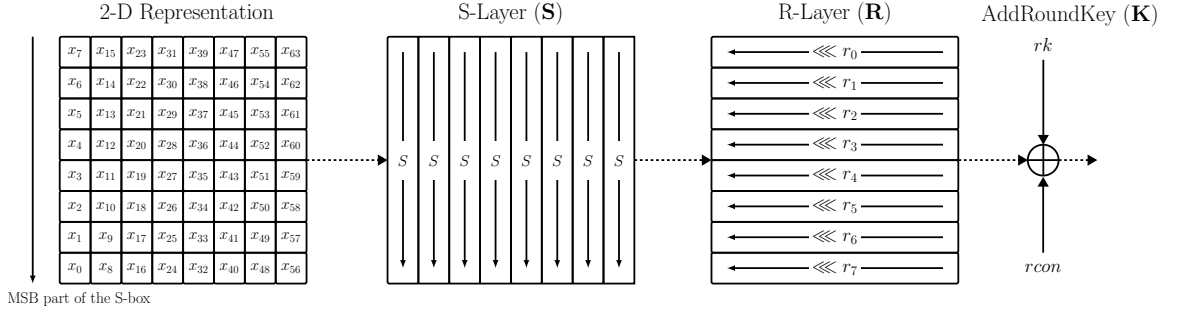


Fig. 1: Round Function of R-Layer based Lightweight Block Cipher

2.1 R-Layer-based Lightweight Block Cipher

Lightweight block ciphers oriented for bit-sliced implementation typically share similar structures. Due to the difficulty of constructing large S-boxes with low implementation and masking costs in a bit-sliced manner, these ciphers often derive larger S-boxes by combining smaller ones. For the linear layer, simple rotation operations are employed to minimize software implementation costs. Notable examples of ciphers following this design paradigm include FLY, PIPO, and other designs. In this paper, we define a 64-bit block cipher as an *R-layer-based lightweight block cipher* if its round function is composed of the following three operations in order: S-Layer, R-Layer, and AddRoundKey, as illustrated in Figure 1.

The round function consists of three operations in the following order, as shown in Figure 1: S-Layer, R-Layer, and AddRoundKey.

- S-Layer (S): Nonlinear operation by applying eight parallel 8-bit S-boxes to the internal state.

- **R-Layer (R)**: Linear diffusion operation by rotating the bits in each row to the left, where the rotation amount for the i -th row is denoted by r_i .
- **AddRoundKey (K)**: Operation by applying the XOR of the round key rk and the round constant $rcon$ to the internal state.

Additionally, a whitening key is applied through an initial XOR operation before the first round, which is regarded as the 0-th round. The single S-box operation used within the S-Layer is denoted by S . The internal state, which is arranged as a two-dimensional array as illustrated in Figure 1, is represented as a one-dimensional sequence as follows:

$$X = (x_0x_1 \dots x_7 \| x_8x_9 \dots x_{15} \| \dots \| x_{56}x_{57} \dots x_{63}),$$

where $x_i \in \{0, 1\}$ for $0 \leq i < 64$.

When convenient, we group every 8 bits into a byte and denote each byte by $X[i]$ for $0 \leq i < 8$, where each $X[i]$ consists of $(x_{8i}, x_{8i+1}, \dots, x_{8i+7})$, and denote the concatenation of bytes $X[i], X[j], \dots, X[k]$ by $X[i, j, \dots, k]$.

2.2 Specification of PIPO and FLY

2.2.1 PIPO

PIPO, proposed at ICISC 2020 [6], is a 13/17-round R-layer-based lightweight block cipher supporting a 64-bit block size and either a 128- or 256-bit master key. These two variants are referred to as PIPO-64-128 and PIPO-64-256, respectively. In PIPO-64-128, the 128-bit master key K is divided into two 64-bit halves, $K = K_0 \| K_1$, and the round key $K_{r \bmod 2}$ is used in the r -th round. For PIPO-64-256, the 256-bit master key K is divided into four 64-bit quarters, $K = K_0 \| K_1 \| K_2 \| K_3$, and $K_{r \bmod 4}$ is used accordingly. A round constant is XORed with the round key before being added to the state. The R-layer performs left rotations with rotation amounts defined as $(r_0, r_1, \dots, r_7) = (0, 7, 4, 3, 6, 5, 1, 2)$. Although a new key schedule aimed at improving resistance to related-key attacks was proposed in [13], it is not considered in this paper. The S-box used in PIPO is detailed in the Appendix A1.

2.2.2 FLY

FLY, proposed in [5], is an R-layer-based lightweight block cipher supporting a 64-bit block size and a 128-bit master key. It features two types of key schedules – one designed to resist related-key attacks and another without such considerations. Since this paper does not focus on related-key attacks, only the simpler key schedule is described. The 128-bit master key K is divided into two 64-bit halves, $K = K_0 \| K_1$, and the round key $K_{r \bmod 2}$ is used alternately in each round. As in PIPO, a round constant is XORed with the round key before being added to the state. The R-layer performs left rotations with rotation amounts defined as $(r_0, r_1, \dots, r_7) = (7, 6, 5, 4, 3, 2, 1, 0)$, and the S-box is detailed in the Appendix A2.

2.3 Differential Cryptanalysis

Differential cryptanalysis was first proposed by Biham and Shamir [7]. This technique exploits the fact that, for a block cipher $E_K(x)$, the output difference (∇) resulting from a specific input difference (Δ) may exhibit statistical behavior that deviates from that of a random permutation. Specifically, if

$$\Pr[\Delta \xrightarrow{E_K} \nabla]$$

is significantly greater than 2^{-n} , where n is the block size, the pair (Δ, ∇) is a *differential distinguisher*. To describe this concept more precisely, we introduce the definition of differential probability.

Definition 1 (Differential Probability) Let $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ be a function. The differential probability $\text{DP}(\Delta \rightarrow \nabla)$ of a transition $\Delta \rightarrow \nabla$ is defined as

$$\text{DP}(\Delta \rightarrow \nabla) := |\{x \in \mathbb{F}_2^n \mid f(x) \oplus f(x \oplus \Delta) = \nabla\}| / 2^n.$$

In practice, it is infeasible to compute the exact differential probability. Instead, the differential probability is estimated through the expected differential probability (EDP) based on the stochastic equivalence hypothesis [14]. The expected differential probability $\text{EDP}(\delta_0 \rightarrow \delta_r)$ is defined as

$$\text{EDP}(\delta_0 \rightarrow \delta_r) := \mathbb{E}_k [\text{DP}(\delta_0 \rightarrow \delta_r)],$$

where the expectation is taken over all possible keys.

Thus, by the stochastic equivalence hypothesis [14], the following approximation is obtained:

$$\text{DP}(\delta_0 \rightarrow \delta_r) \approx \text{EDP}(\delta_0 \rightarrow \delta_r).$$

However, in most cases, it is still difficult to compute the expected differential probability exactly. Therefore, attackers typically search for a dominant expected differential characteristic and use it to estimate the differential probability.

Definition 2 (Expected Differential Characteristic) Let E_K be an r -round iterated block cipher, represented as

$$E_K(\cdot) = f_r \circ f_{r-1} \circ \cdots \circ f_1(\cdot).$$

An expected differential characteristic is a sequence of intermediate differences defined at each round. Under the Markov cipher assumption [14], the expected probability of a differential characteristic is computed as the product of the expected differential probabilities of each round:

$$\text{EDP}(\delta_0, \delta_1, \dots, \delta_r) = \prod_{i=1}^r \mathbb{E}_k [\text{DP}(\delta_{i-1} \rightarrow \delta_i)].$$

In some cases, a tighter estimation can be achieved by considering not only a single dominant expected characteristic but also multiple expected differential characteristics

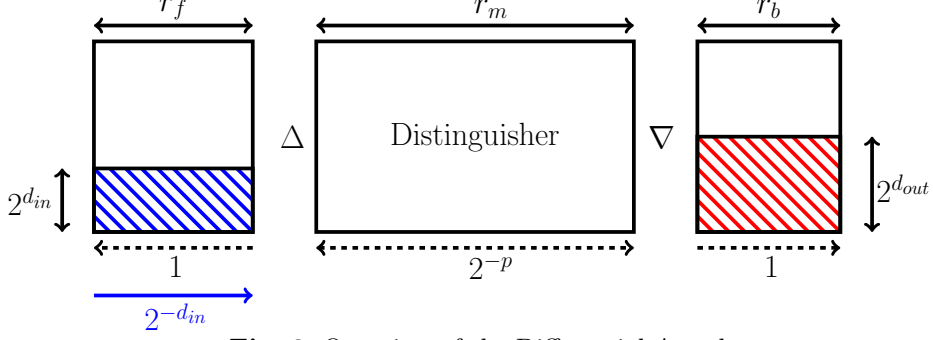


Fig. 2: Overview of the Differential Attack

that share the same input and output differences. This approach is referred to as the clustering effect.

Definition 3 (Clustering Effect) Let E_K be an r -round iterated block cipher, where

$$E_K(\cdot) = f_r \circ f_{r-1} \circ \cdots \circ f_1(\cdot).$$

The differential probability $\text{DP}(\delta_0 \rightarrow \delta_r)$ is then approximated as

$$\text{DP}(\delta_0 \rightarrow \delta_r) \approx \sum_{\substack{c_0=\delta_0 \\ c_r=\delta_r}} \prod_{i=1}^r \text{EDP}(c_{i-1} \rightarrow c_i),$$

To simplify notation, it is convenient to represent differential probabilities in terms of differential weights.

Definition 4 (Differential Weight) The differential weight of a transition $\text{DP}[\Delta \rightarrow \nabla]$ is defined as the negative binary logarithm of the differential probability:

$$W[\Delta \rightarrow \nabla] = -\log_2(\text{DP}[\Delta \rightarrow \nabla]).$$

3 Revisiting Previous Key-Recovery Attack

In this section, we revisit the differential key-recovery framework outlined in [6], along with the methodology for computing key-recovery complexity. As illustrated in Fig. 2, given a differential (Δ, ∇) covering r_m rounds with probability 2^{-p} , we construct a key-recovery attack by prepending r_f rounds and appending r_b rounds. The attack proceeds in three main phases.

Notations

The following notations are used throughout this paper:

- $E_\kappa(\cdot)$: The encryption function under key κ , defined as $E(\kappa, \cdot)$.
- $E_{\kappa_{\text{in}}}^{\text{in}}(\cdot)$: The partial encryption function with the subkey κ_{in} .

Algorithm 1: Overview of the Differential Attack Framework in [6]

```

1 Phase 1 : Data Collection
2   for structure in structures do
3     Encrypt plaintexts derived from the structure and store them in the
       hash table for the attack.
4 Phase 2 : Matching and Filtering
5   for  $\kappa_{in} \in K_{in}$  do
6     Retrieve pairs from the hash table that satisfy the input difference
       ( $\Delta$ ), and construct  $\mathcal{C}$ .
7     Phase 3 : Key Counting
8     for  $(C, \tilde{C}) \in \mathcal{C}$  do
9       for  $\kappa_{out} \in K_{out}$  do
10      Apply the key schedule and the early-abort technique to
        check whether the (truncated) differential condition holds
        S-box-wise. If the output difference ( $\nabla$ ) is satisfied,
        increment the key counter corresponding to  $(\kappa_{in} \parallel \kappa_{out})$ 

```

- $E_K(\cdot)$: The encryption oracle available to the adversary under the CPA assumption.
- K_m : The size of the master key in bits.
- CP: A chosen plaintext, considered as a unit of data.
- D_{in} : Set of input differences propagating backward from Δ through r_f rounds.
- D_{out} : Set of output differences propagating forward from ∇ through r_b rounds.
- K_{in} : Set of subkeys that be guessed to derive Δ from D_{in} .
- K_{out} : Set of subkeys that be guessed to derive ∇ from D_{in} .
- d_*, k_* : $\log_2(\|D_*\|)$ and $\log_2(\|K_*\|)$, respectively.
- Right pair: A pair $(P, \tilde{P})$ such that $P \oplus \tilde{P} = \Delta$ and the corresponding ciphertexts satisfy $C \oplus \tilde{C} = \nabla$, where $C = E_K(P)$ and $\tilde{C} = E_K(\tilde{P})$.
- μ : Number of right pairs.
- c_e : The number of S-box evaluations per encryption.
- $\#AS(\Delta)$: Number of active (non-zero difference) S-boxes in the S-layer for input difference Δ .
- N_{Δ, r_f} : The number of S-boxes that need to be guessed to derive Δ from D_{in} .
- M_{∇, r_b} : The number of S-boxes that need to be guessed to derive ∇ from D_{in} .
- $\Lambda(\mu, p, d_{in})$: Defined as

$$\Lambda(\mu, p, d_{in}) = \begin{cases} \mu \cdot 2^{p+1}, & \text{if } \log_2 \mu + p \geq d_{in} - 1, \\ 2^{d_{in}}, & \text{otherwise.} \end{cases}$$

For simplicity, $\Lambda(\mu, p, d_{in})$ may be denoted by Λ when no confusion arises.

3.1 Phase 1: Data Collection

In this phase, the adversary performs encryption operations to obtain μ right pairs using the given differential distinguisher. To efficiently generate these pairs, a technique commonly referred to as a *structure* is employed. A structure is constructed by assigning all possible values to the active S-boxes determined by D_{in} in the input while fixing the inactive parts.

The number of pairs generated by a single structure is $2^{2d_{\text{in}}-1}$, and each pair has a probability of $2^{-d_{\text{in}}}$ to satisfy the input difference Δ . Thus, each structure yields $2^{d_{\text{in}}-1}$ pairs matching the input difference. By employing 2^s structures, a total of $2^{s+d_{\text{in}}-1}$ valid pairs can be obtained. To acquire μ right pairs satisfying the given differential trail, $\mu 2^p$ pairs are required, resulting in:

$$s = p - d_{\text{in}} + 1 + \log_2 \mu.$$

However, if $2^{d_{\text{in}}-1}$ exceeds the number of required pairs, the adversary cannot arbitrarily reduce the structure size, as a structure must be fully defined over all active bits. As a result, more data than necessary may be collected, leading to inefficiency.

Consequently, the complexities for data collection are given by

$$\begin{aligned} \mathcal{T}_{\text{data}} &= \Lambda(\mu, p, d_{\text{in}}) \quad (\text{encs}), \\ \mathcal{M}_{\text{data}} &= 8 \cdot \Lambda(\mu, p, d_{\text{in}}) \quad (\text{bytes}), \\ \mathcal{D}_{\text{data}} &= \Lambda(\mu, p, d_{\text{in}}) \quad (\text{CP}). \end{aligned}$$

3.2 Phase 2: Matching and Filtering

In this phase, for each $\kappa_{\text{in}} \in K_{\text{in}}$, $\mu \cdot 2^p$ plaintext pairs satisfying the input differential condition Δ must be matched among the plaintext-ciphertext pairs collected during the data collection phase. Additionally, a filtering step is performed based on the output differential condition D_{out} to eliminate invalid pairs at an early stage. Although optional, this filtering significantly improves the overall efficiency of the attack by discarding irrelevant candidates early.

To this end, for each plaintext P , partial encryption ($E_{\kappa_{\text{in}}}^{\text{in}}$) is applied to compute a corresponding $\tilde{P}$ that satisfies the following conditions:

$$E_{\kappa_{\text{in}}}^{\text{in}}(P) \oplus E_{\kappa_{\text{in}}}^{\text{in}}(\tilde{P}) = \Delta, \text{ and } E_K(P) \oplus E_K(\tilde{P}) \in D_{\text{out}}.$$

Then, the following set of ciphertext pairs is constructed:

$$\mathcal{C} := \{(C, \tilde{C}) : C = E_K(P), \tilde{C} = E_K(\tilde{P})\},$$

where $\|\mathcal{C}\| = \mu \cdot 2^{p-64+d_{\text{out}}}$.

This process requires two accesses to a hash table that stores Λ ciphertexts and incurs $\|\mathcal{C}\| \cdot N_{\Delta, r_f}$ S-box operations. Since the table is large, each access is assumed to cost the same as one full encryption. Accordingly, the time complexity of the matching

Algorithm 2: Early-Abort Key Counting Algorithm with a Linear Key Schedule

Input: Filtered pair set $\mathcal{P}$, key sets $K_0, K_1, \dots, K_{N-1}$, and truncated conditions $c_0, c_1, \dots, c_{N-1}$

```

1 for pair  $P$  in  $\mathcal{P}$  do
2   for  $k_0 \in K_0$  do
3     if  $k_0$  does not satisfy  $c_0$  then
4       break
5     for  $k_1 \in K_1 \setminus (K_0 \cap K_1)$  do
6       if  $k_1$  does not satisfy  $c_1$  then
7         break
8        $\vdots$ 
9     for  $k_{N-1} \in K_{N-1} \setminus \bigcup_{i=0}^{N-2} (K_i \cap K_{N-1})$  do
10      if  $k_{N-1}$  does not satisfy  $c_{N-1}$  then
11        break
12      Increase key counter  $\text{cnt}[k_0 \| k_1 \| \dots \| k_{N-1}]$ 

```

phase in the key recovery process is given by:

$$\mathcal{T}_{\text{mf}} = 2^{k_{\text{in}}} \cdot \mu 2^{p+1-64+d_{\text{out}}} \cdot (1 + c_e^{-1} \cdot N_{\Delta, r_f}) \quad (\text{encs}).$$

The memory complexity for storing the matched ciphertext pairs is:

$$\mathcal{M}_{\text{mf}} = 8 \cdot \mu \cdot 2^{p+1-64+d_{\text{out}}} \quad (\text{bytes}).$$

3.3 Phase 3 : Key Counting

Finally, each ciphertext pair is verified to determine whether it satisfies the given output difference ∇ . To accomplish this, as described in Algorithm 2, round keys corresponding to the S-boxes through which the r_b -round truncated differential trail passes at the output side are guessed, and the pairs are checked against the specified truncated differential condition.

Considering the linear key schedule and truncated differential, the expected computational complexity for key counting per pair is given by:

$$\begin{aligned}
\mathcal{T}_{\text{kc}} &= \frac{1}{c_e} \cdot 2^{k_{\text{in}}} \cdot \mu 2^{p+1-64+d_{\text{out}}} \cdot \sum_{i=0}^{M_{\nabla, r_b}-1} 2^{\sum_{j=0}^i (g(j)-f(j))} \\
&= \frac{1}{c_e} \cdot \mu \cdot 2^{p-63+k_{\text{in}}+d_{\text{out}}} \cdot \sum_{i=0}^{M_{\nabla, r_b}-1} 2^{\sum_{j=0}^i (g(j)-f(j))} \quad (\text{encs}),
\end{aligned}$$

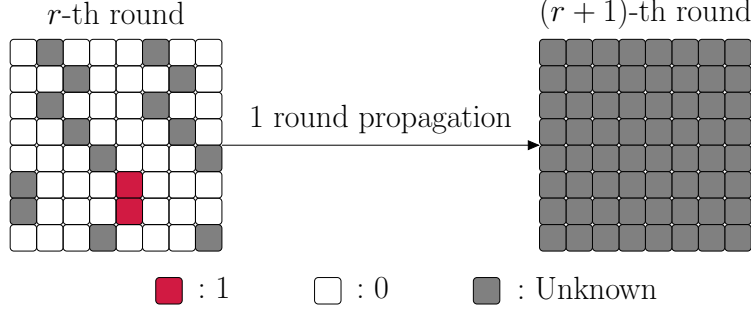


Fig. 3: Example of key counting

$$\mathcal{M}_{\text{kc}} = 2^{k_{\text{in}} + k_{\text{out}}} \quad (\text{bytes}),$$

where $g(j)$ denotes the number of key bits guessed for the j -th S-box through which the differential trail passes, and $f(j)$ denotes the average number of key candidates per pair that do not satisfy the prescribed differential condition at step j . In particular, $f(0)$ is defined to be 0.

Example 1 As an example of key counting, we consider the case where the r -th and $(r+1)$ -th round output differences are as shown in Fig. 3. In a single iteration of **Phase 3** in Algorithm 1, we need to verify whether the r -th round output difference is satisfied by guessing the subkey and performing S-box-wise evaluation using the $(r+1)$ -th round outputs, according to Algorithm 2.

To compute the output difference of the 0-th S-box, we first guess the corresponding 8-bit subkey (after applying the inverse of the R-layer). Then, we check whether the output difference satisfies the condition $0**00000$. If the condition is not met, the process is immediately aborted (early-abort) as described in Algorithm 2. Although 8 bits are guessed in total, only about 2^2 key candidates satisfy the output difference condition on average. Thus, it can be regarded that 6 bits are filtered out at this step.

This procedure is repeated for each S-box from index 0 to 7. The overall complexity can be expressed as:

$$\sum_{i=0}^7 2^{\sum_{j=0}^i (g(j) - f(j))} = 2^{20.481},$$

where

$$g = [8, 8, 8, 8, 8, 8, 8, 8], \quad f = [0, 6, 6, 6, 6, 8, 6, 6].$$

An observation is that the total complexity depends on the order in which the S-boxes are processed. By reordering them to minimize $g(j) - f(j)$ at each step, we obtain:

$$g_{\text{ord}} = [g(5), g(0), g(1), g(2), g(3), g(4), g(6), g(7)],$$

$$f_{\text{ord}} = [0, f(5), f(0), f(1), f(2), f(3), f(4), f(6)].$$

Accordingly, the total complexity after reordering becomes:

$$\sum_{i=0}^7 2^{\sum_{j=0}^i (g_{\text{ord}}(j) - f_{\text{ord}}(j))} = 2^{20.415}.$$

3.4 Complexity and Success Probability

Thus, the overall complexity of the key recovery attack can be summarized as follows:

Time complexity

$$\begin{aligned} \mathcal{T} &= \mathcal{T}_{\text{data}} + \mathcal{T}_{\text{mf}} + \mathcal{T}_{\text{kc}} + 2^{K_m - a} \\ &= \mu 2^{p-63+k_{\text{in}}+d_{\text{out}}} \cdot \left\{ 1 + \frac{1}{c_e} \left(N_{\Delta, r_f} + \sum_{i=0}^{M_{\nabla, r_b} - 1} 2^{\sum_{j=0}^i (g(j) - f(j))} \right) \right\} \\ &\quad + \Lambda + 2^{K_m - a} \quad (\text{encs}). \end{aligned} \quad (1)$$

where a denotes the advantage obtained through key ranking, assuming that the right key is positioned within the top r candidates. If the subkey size is K_{sub} bits, this advantage can be quantified as $a = K_{\text{sub}} - \log_2 r$. Accordingly, instead of exhaustively testing all $2^{K_{\text{sub}}}$ candidates, the attacker is expected to recover the key by checking only the top r candidates, thus reducing the overall complexity by a factor of 2^a .

Data complexity

$$\mathcal{D} = \mathcal{D}_{\text{data}} = \Lambda(\mu, p, d_{\text{in}}) \quad (\text{CP}). \quad (2)$$

Memory complexity

$$\begin{aligned} \mathcal{M} &= \mathcal{M}_{\text{data}} + \mathcal{M}_{\text{mf}} + \mathcal{M}_{\text{kc}} \quad (\text{bytes}) \\ &= 8 \cdot (\mu 2^{p-64+d_{\text{out}}} + \Lambda) + 2^{k_{\text{in}}+k_{\text{out}}} \quad (\text{bytes}) \end{aligned} \quad (3)$$

The success probability of the differential attack can then be calculated using Theorem 1.

Theorem 1 ([15]) Let $\mathbb{P}_S$ be the success probability of a differential attack. Then $\mathbb{P}_S$ is calculated as follows:

$$\mathbb{P}_S = \Phi \left(\frac{\sqrt{\mu S_N} - \Phi^{-1}(1 - 2^{-a})}{\sqrt{S_N + 1}} \right), \quad (4)$$

where a is the advantage, Φ is the cumulative normal distribution, μ is the number of right pairs, and S_N is the signal-to-noise ratio.

4 Searching Key-Recovery-Attack Friendly Distinguisher

In most block cipher analyses, differential distinguishers are typically selected based on the maximum expected differential probability. However, this criterion is not always suitable for key-recovery attacks, where the number of key bits that must be guessed often has a greater impact on the overall attack complexity than the differential probability itself [10]. To address this, we focus on distinguishers that involve fewer key bits during the key guessing phase. These are referred to as *key-recovery-attack-friendly distinguishers* [10].

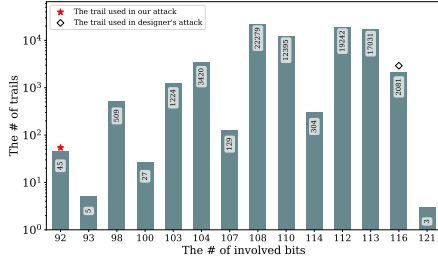


Fig. 4: Distribution of differential trails for PIPO

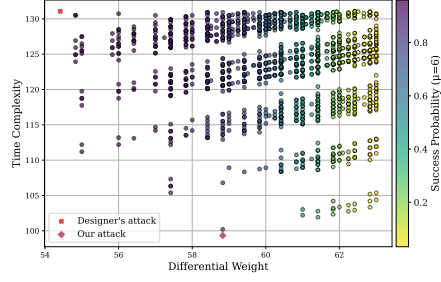


Fig. 5: Attack complexity and success probability for PIPO

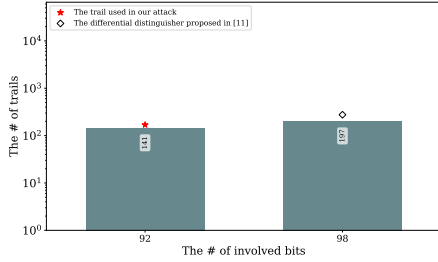


Fig. 6: Distribution of differential trails for FLY

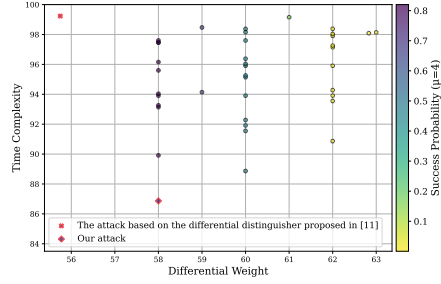


Fig. 7: Attack complexity and success probability for FLY

To identify such distinguishers, we adopted the accelerated version of Matsui's search algorithm proposed by Kim et al. [12]. This method is significantly faster than approaches that encode Matsui's algorithm into off-the-shelf tools such as MILP or SAT solvers [16–19], making it well suited for large-scale trail enumeration. Moreover, it enables the exhaustive search of all differential trails above a given probability threshold while accounting for the clustering effect of fixed input-output differences.

Using this approach, we searched all differential trails with probability at least 2^{-63} and identified those involving fewer key bits, which are more key-recovery-attack friendly.

4.1 Searching for PIPO Distinguisher

For PIPO, the designers searched and presented the maximum expected differential characteristics. They analyzed all input-output pairs with the maximum expected differential characteristics and applied the clustering effect to each pair to propose distinguishers and conduct differential attacks. Meanwhile, we analyzed all possible expected differential characteristics to devise an attack with superior key recovery complexity compared to the attack proposed by the designers. Specifically, we utilized

a branch-and-bound algorithm to thoroughly search all 6-round differential trails with probabilities of 2^{-63} or higher. Through this process, we identified a total of 78,693 trails with improved time complexity compared to those proposed by the designers. Additionally, Fig. 4 shows the distribution of differential trails based on the number of key bits involved in the key recovery phase. While the distinguisher proposed by the designers required 116 involved key bits, the most key-recovery-friendly distinguisher was identified with only 92 involved key bits. The key recovery complexity and success probability for each of the 78,693 distinguishers are also presented in Fig. 5.

4.2 Searching for FLY Distinguisher

We applied the same methodology to FLY and exhaustively analyzed all 8-round differential trails with probabilities of at least 2^{-63} . As a result, we identified 337 trails, among which several exhibited better key-recovery complexity than the distinguisher proposed in [11]. Fig. 6 shows the distribution of trails based on the number of involved key bits. While the distinguisher in [11] requires 98 key bits, our best distinguisher involves only 92 bits. The key-recovery complexity and success probability for all 337 distinguishers are illustrated in Fig. 7.

5 Key Recovery Attacks on PIPO

In this section, we present concrete key-recovery attacks constructed based on the identified optimal differential distinguishers. For PIPO-64-128, we propose an improved key-recovery attack compared to the previously known result in [6]. For PIPO-64-256, while only the number of attackable rounds had been reported in [6] without concrete key-recovery analyses, we construct a 13-round key-recovery attack in a full-codebook setting to demonstrate its feasibility. Furthermore, to eliminate the need for a full codebook, we employ a time-data-memory trade-off attack based on the imposed condition technique introduced in [20].

To facilitate the description of our key-recovery procedures and complexity evaluations, we define the following notations:

- $\Delta \mathbf{S}^r$, $\Delta \mathbf{R}^r$, and $\Delta \mathbf{K}^r$: Differences at the output of the r -th round's S-layer, R-layer, and AddRoundKey operations, respectively.
- S^{-1} , $\mathbf{S}^{-1}$, and $\mathbf{R}^{-1}$: The inverse S-box, inverse S-layer, and inverse R-layer, respectively.
- $RK_r^\oplus$: The state of $RK_r \oplus c_r$, where RK_r and c_r denote the r -th round key and the corresponding round constant.
- $\overline{RK_r^\oplus}$: $\mathbf{R}^{-1}(RK_r^\oplus)$, representing the equivalent key obtained by inverting the R-layer.
- $x_{\mathbf{S}}^r$: The input value to the S-Layer at round r .
- $x_{\mathbf{R}}^{r-1}$: The value calculated by $\mathbf{R}^{-1}(x_{\mathbf{S}}^r)$ (due to the use of the equivalent key representation).

The details of each attack are provided in the following subsections.

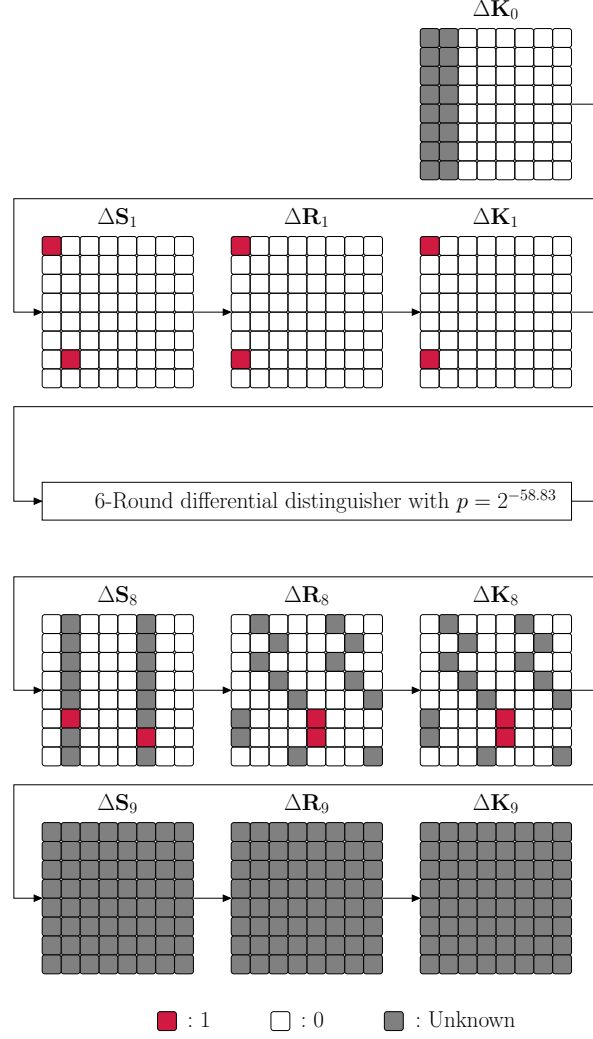


Fig. 8: 9-Round key-recovery attack on PIPO-64-128

5.1 Key-Recovery Attack on 9-Round PIPO-64-128

We describe our key-recovery attack on 9-round PIPO-64-128 through the following steps, as illustrated in Fig. 8.

1. Select random values for $P[2], \dots, P[7]$, and assign all 2^{16} possible values to $P[0, 1]$. Then, encrypt these plaintexts and store the corresponding ciphertexts in a hash table.
2. Guess the 16-bit subkey $RK_0[0, 1]$, i.e., $K_{in} = RK_0[0, 1]$.

3. For each $\kappa_{\text{in}} \in K_{\text{in}}$, match 2^{15} pairs $(P, \tilde{P})$ in the hash table that satisfy

$$E_{\kappa_{\text{in}}}^{\text{in}}(P) \oplus E_{\kappa_{\text{in}}}^{\text{in}}(\tilde{P}) = \Delta \mathbf{S}_1,$$

and construct the set

$$\mathcal{C} := \{(C, \tilde{C}) : C = E_K(P), \tilde{C} = E_K(\tilde{P})\},$$

where $\|\mathcal{C}\| = 2^{15}$.

4. Initialize a 2^{92} key counter $\text{cnt}[\cdot]$ corresponding to K_{in} and K_{out} .
5. For each ciphertext pair $(C, \tilde{C}) \in \mathcal{C}$, compute $(x_{\mathbf{R}}^9, \tilde{x}_{\mathbf{R}}^9) = (\mathbf{R}^{-1}(C), \mathbf{R}^{-1}(\tilde{C}))$.
6. Guess an 8-bit subkey $\overline{RK_9^\oplus}[4]$. For each candidate, compute

$$x_{\mathbf{S}}^9[4] = S^{-1} \left(x_{\mathbf{R}}^9[4] \oplus \overline{RK_9^\oplus}[4] \right),$$

$$\tilde{x}_{\mathbf{S}}^9[4] = S^{-1} \left(\tilde{x}_{\mathbf{R}}^9[4] \oplus \overline{RK_9^\oplus}[4] \right),$$

and verify whether

$$x_{\mathbf{S}}^9[4] \oplus \tilde{x}_{\mathbf{S}}^9[4] = \Delta \mathbf{K}^8[4].$$

7. Similarly, for each remaining byte $\overline{RK_9^\oplus}[0, 1, 2, 3, 5, 6, 7]$, guess the corresponding subkey, apply the inverse S-box individually, and verify the output differences $\Delta \mathbf{K}^8[0, 1, 2, 3, 5, 6, 7]$.
8. If all subkey bytes satisfy the conditions in Round 9, compute $(x_{\mathbf{R}}^8, \tilde{x}_{\mathbf{R}}^8) = (\mathbf{R}^{-1}(x_{\mathbf{S}}^9), \mathbf{R}^{-1}(\tilde{x}_{\mathbf{S}}^9))$.
9. Using the key schedule, 2 bits of each $\overline{RK_8^\oplus}[i]$ ($0 \leq i \leq 7$) can be determined from the initial guess $\overline{RK_0}[0, 1]$. The remaining 6 bits of each subkey byte $\overline{RK_8^\oplus}[1]$ and $\overline{RK_8^\oplus}[5]$ are subsequently guessed. The inverse S-boxes are then applied to verify whether the resulting differences match the expected values $\Delta \mathbf{K}^7[1, 5]$ ($= \nabla[1, 5]$).
10. If all conditions are satisfied, increment the corresponding 92-bit key counter:

$$\text{cnt} \left[\underbrace{\overline{RK_0}[0, 1] \parallel \overline{RK_9^\oplus}[0, 1, \dots, 7]}_{80\text{-bit}} \parallel \underbrace{\overline{RK_8^\oplus}[1, 5]}_{12\text{-bit}} \right]$$

11. Repeat steps 1 to 10, $2^{46.415}$ times.

For the detailed values of $\Delta \mathbf{S}^r$, $\Delta \mathbf{R}^r$, and $\Delta \mathbf{K}^r$, can be found in Table B6 of Appendix B.

5.1.1 Complexity Analysis

As analyzed in the previous section, the attack consists of successive steps of guessing and filtering in key counting. Table 2 summarizes the parameters used in this process, and Table 3 presents the values at each step of guessing and filtering.

Table 2: Parameter settings for the 11-round PIPO attack

Symbol	Value
μ	6
p	58.83
N_{Δ, r_f}	2
M_{∇, r_b}	10
$\Lambda(\mu, p, d_{\text{in}})$	$6 \times 2^{58.83+1}$
d_{in}	16
d_{out}	64
k_{in}	16
k_{out}	76
c_e	$2^{6.17}$
K_m	128
a	38

Table 3: Successive steps of guessing and filtering in key counting for the 11-round PIPO attack

i	$g(i)$	$f(i)$	$\sum_{j=0}^i (g(j) - f(j))$
0	8	0	8
1	8	8	8
2	8	6	10
3	8	6	12
4	8	6	14
5	8	6	16
6	8	6	18
7	8	6	20
8	6	6	20
9	6	7	19

Accordingly, the overall time, data, and memory complexities and the success probability are given by Eqs. 1–4 as follows:

$$\begin{aligned} \mathcal{T} \approx & 6 \times 2^{58.83+1+16} \cdot \{1 + 2^{-6.17} \cdot (2 + 2^8 + \dots + 2^{19})\} \\ & + 6 \times 2^{58.83+1} + 2^{128-38} = 2^{93.851} \quad (\text{encs}). \end{aligned}$$

$$\begin{aligned}
\mathcal{D} &= 6 \times 2^{58.83+1} = 2^{62.415} \quad (\text{CP}). \\
\mathcal{M} &= 8 \cdot 2 \cdot 2^{16} + 2^{16+76} \approx 2^{92} \quad (\text{bytes}). \\
\mathbb{P}_S &= 0.9 \quad (90\%).
\end{aligned}$$

5.2 Key-Recovery Attack on 13-Round PIPO-64-256

We propose a 13-round key-recovery attack on PIPO-64-256 by exploiting the linearity of the key schedule. As illustrated in Fig. C1, 3 rounds are added at the beginning and 4 rounds at the end for key recovery, and the attack is conducted based on this distinguisher. The following describes the attack procedure:

1. Assign all 2^{64} values for $P[0, \dots, 7]$. Then, encrypt these plaintexts and store the corresponding ciphertexts in a hash table.
2. Guess the 64-bit subkey $RK_0[0, 1, \dots, 7]$.
3. Guess the 64-bit subkey $RK_1[0, 1, \dots, 7]$.
4. Guess the 16-bit subkey $RK_2[0, 1]$, i.e., $K_{\text{in}} = RK_0 \times RK_1 \times RK_2[0, 1]$.
5. For each $k \in K_{\text{in}}$, match $6 \cdot 2^{58.83}$ pairs $(P, \tilde{P})$ in the hash table that satisfy

$$E_{\kappa_{\text{in}}}^{\text{in}}(P) \oplus E_{\kappa_{\text{in}}}^{\text{in}}(\tilde{P}) = \Delta \mathbf{S}_3,$$

and construct the set

$$\mathcal{C} := \{(C, \tilde{C}) : C = E_K(P), \tilde{C} = E_K(\tilde{P})\},$$

where $\|\mathcal{C}\| = 6 \cdot 2^{58.83}$.

6. Initialize a 2^{220} key counter $\text{cnt}[\cdot]$ corresponding to K_{in} and K_{out} .
7. For each ciphertext pair $(C, \tilde{C}) \in \mathcal{C}$, compute $(x_{\mathbf{R}}^{13}, \tilde{x}_{\mathbf{R}}^{13}) = (\mathbf{R}^{-1}(C), \mathbf{R}^{-1}(\tilde{C}))$.
8. From the key schedule, all bits of $\overline{RK_{13}^{\oplus}}$ are already determined from the initial guess RK_1 . Subsequently, compute $x_{\mathbf{S}}^{13} = \mathbf{S}^{-1}(x_{\mathbf{R}}^{13} \oplus \overline{RK_{13}^{\oplus}})$ and $\tilde{x}_{\mathbf{S}}^{13} = \mathbf{S}^{-1}(\tilde{x}_{\mathbf{R}}^{13} \oplus \overline{RK_{13}^{\oplus}})$.
9. Compute $(x_{\mathbf{R}}^{12}, \tilde{x}_{\mathbf{R}}^{12}) = (\mathbf{R}^{-1}(x_{\mathbf{S}}^{13}), \mathbf{R}^{-1}(\tilde{x}_{\mathbf{S}}^{13}))$.
10. Similarly, all bits of $\overline{RK_{12}^{\oplus}}$ are already determined from the initial guess RK_0 . Subsequently, compute $x_{\mathbf{S}}^{12} = \mathbf{S}^{-1}(x_{\mathbf{R}}^{12} \oplus \overline{RK_{12}^{\oplus}})$ and $\tilde{x}_{\mathbf{S}}^{12} = \mathbf{S}^{-1}(\tilde{x}_{\mathbf{R}}^{12} \oplus \overline{RK_{12}^{\oplus}})$.
11. Compute $(x_{\mathbf{R}}^{11}, \tilde{x}_{\mathbf{R}}^{11}) = (\mathbf{R}^{-1}(x_{\mathbf{S}}^{12}), \mathbf{R}^{-1}(\tilde{x}_{\mathbf{S}}^{12}))$.
12. Guess an 8-bit subkey $\overline{RK_{11}^{\oplus}}[4]$, apply the inverse S-box to $x_{\mathbf{R}}^{11}[4] \oplus \overline{RK_{11}^{\oplus}}[4]$ and $\tilde{x}_{\mathbf{R}}^{11}[4] \oplus \overline{RK_{11}^{\oplus}}[4]$, and check whether the output difference matches $\Delta \mathbf{K}^{10}[4]$.
13. Similarly, for each remaining byte $\overline{RK_{11}^{\oplus}}[0, 1, 2, 3, 5, 6, 7]$, guess the corresponding subkey, apply the inverse S-box, and verify whether the resulting differences match the expected values $\Delta \mathbf{K}^{10}[0, 1, 2, 3, 5, 6, 7]$.
14. If all subkey bytes satisfy the conditions in round 11, compute

$$(x_{\mathbf{R}}^{10}, \tilde{x}_{\mathbf{R}}^{10}) = (\mathbf{R}^{-1}(x_{\mathbf{S}}^{11}), \mathbf{R}^{-1}(\tilde{x}_{\mathbf{S}}^{11})).$$

15. Guess the remaining 6 bits of each subkey byte $\overline{RK}_{10}^{\oplus}[1]$ and $\overline{RK}_{10}^{\oplus}[5]$. Then, apply the inverse S-boxes and verify whether the resulting differences match the expected values $\Delta\mathbf{K}^9[1, 5]$.
16. If all conditions are satisfied, increment the corresponding 220-bit key counter:

$$\text{cnt} \left[\underbrace{RK_0[0, \dots, 7] \parallel \dots \parallel RK_2[0, 1]}_{144\text{-bit}} \parallel \underbrace{\overline{RK}_{11}^{\oplus}[0, \dots, 7] \parallel \overline{RK}_{10}^{\oplus}[1, 5]}_{76\text{-bit}} \right]$$

For the detailed values of $\Delta\mathbf{S}^r$, $\Delta\mathbf{R}^r$, and $\Delta\mathbf{K}^r$, can be found in Table C7 of Appendix C.

5.2.1 Complexity Analysis

The parameters used in the attack and the values obtained at each step of successive guessing and filtering are summarized in Table C8 in Appendix C.

Accordingly, the overall complexities and the success probability are given by Eqs. 1–4 as follows:

$$\begin{aligned} \mathcal{T} &\approx 6 \cdot 2^{58.83+1+144} \cdot \{1 + 2^{-6.17} \cdot (18 + 1 + \dots + 2^{19})\} \\ &\quad + 2^{64} + 2^{256-38} = 2^{221.851} \quad (\text{encryptions}). \\ \mathcal{M} &= 8 \cdot (6 \cdot 2^{58.83} + 2^{64}) + 2^{144+76} \approx 2^{220} \quad (\text{bytes}). \\ \mathcal{D} &= 2^{64} \quad (\text{CP}). \\ \mathbb{P}_S &= 0.9 \quad (90\%). \end{aligned}$$

5.2.2 Time–Data–Memory Trade-offs to Avoid Full Codebook

To avoid using a full codebook our attack when all input bits are active, we adopt the *imposed condition* approach proposed in [20]. The procedure is as follows:

1. Assign 2^{64-x} values to plaintexts P , where x bits are fixed. Then encrypt these plaintexts and store the corresponding ciphertexts in a hash table.
2. Guess 144 bits of $k \in K_{\text{in}}$ ($= RK_0 \times RK_1 \times RK_2[0, 1]$).
3. For each $k \in K_{\text{in}}$, retrieve $2^x \times 6 \cdot 2^{58.83}$ pairs $(P, \tilde{P})$ from the hash table satisfying

$$E_k^{\text{in}}(P) \oplus E_k^{\text{in}}(\tilde{P}) = \Delta\mathbf{S}_1 \text{ and } \tilde{P} \text{ shares the same fixed } x \text{ bits as } P.$$

Then construct the set

$$\mathcal{C} := \{(C, \tilde{C}) : C = E_K(P), \tilde{C} = E_K(\tilde{P})\},$$

where $\|\mathcal{C}\| = 6 \cdot 2^{58.83}$.

4. From this point onward, the steps proceed identically to the original algorithm.

The value of x is chosen to satisfy the constraint

$$6 \cdot 2^{58.83+x} \leq 2^{64-x},$$

derived from the data complexity bound.

As a result, the time and data complexities are updated as follows:

$$\begin{aligned} \mathcal{T} = & 6 \times 2^{58.83+144+x} \cdot \{1 + 2^{144-6.17} \times (18 + \dots + 2^{19})\} \\ & + 2^{64-x} + 2^{256-38} \quad (\text{encs}), \end{aligned}$$

$$\mathcal{D} = 2^{64-x} \quad (\text{CP}),$$

$$\text{and } \mathcal{M} = 8 \cdot (6 \times 2^{58.83+x} + 2^{64-x}) + 2^{144+76} \quad (\text{bytes}).$$

The success probability remains unchanged. In particular, by setting $x = 1$, the resulting complexities become:

$$\mathcal{T} = 2^{222.403} \quad (\text{encryptions}),$$

$$\mathcal{D} = 2^{63} \quad (\text{CP}), \quad \text{and} \quad \mathcal{M} = 2^{220} \quad (\text{plaintexts}).$$

6 Key-Recovery Attacks on FLY

In this section, we present key-recovery attacks constructed based on the identified optimal differential distinguishers. We first estimate the attack complexity using the distinguisher proposed in [11]. Although no key-recovery attack was explicitly presented in [11], and the focus was solely on the probability of the distinguisher itself, we use it as a reference point to estimate the complexity within our attack framework. According to our analysis, a key-recovery attack using this distinguisher would require approximately $2^{99.523}$ encryptions and is limited to 11 rounds.

In contrast, based on the identified optimal distinguisher for the 11-round attack, a key-recovery attack requiring approximately 2^{72} encryptions, 2^{92} bytes of memory, and $2^{59.77}$ data complexity can be performed. Furthermore, this attack is extended to 12 rounds, which was infeasible when using the distinguisher in [11].

We first present a full-codebook attack. Subsequently, for short key-recovery rounds, we propose an attack that leverages the S-box properties to achieve a better time–data–memory trade-off than the imposed condition approach [20]. The details and procedures of each attack are described in the following subsections, where the notations defined in Section 5 are consistently used.

6.1 Estimating the Key-Recovery Complexity of the Distinguisher in [11]

We first estimate the attack complexity implied by the distinguisher proposed in [11], using our analysis framework. Although a key-recovery attack was not explicitly proposed in [11], the corresponding complexity is estimated to be approximately $2^{99.233}$ encryptions. This estimate serves as a useful reference point for comparison with our

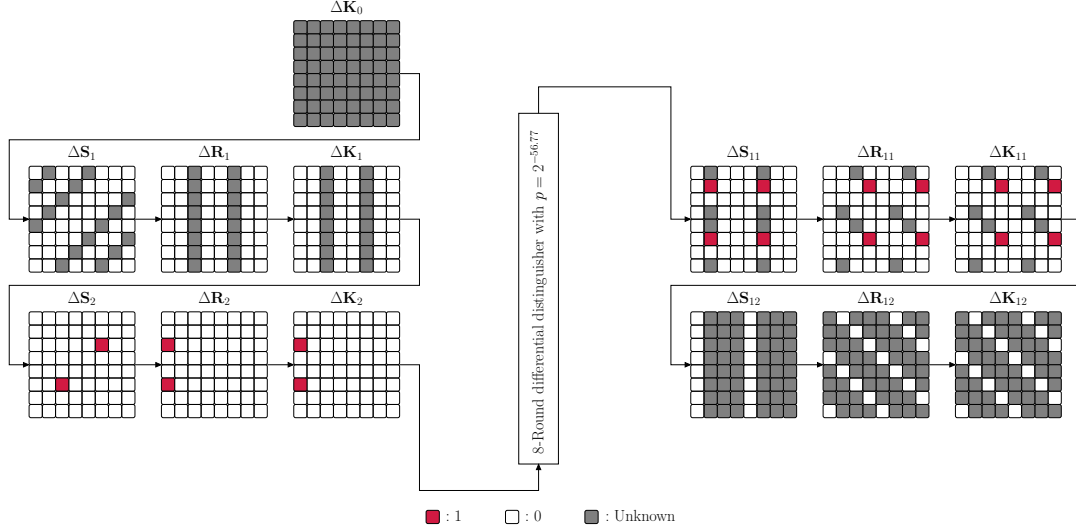


Fig. 9: 12-round key-recovery attack on FLY

improved approach. In contrast, our 11-round attack on FLY using the proposed distinguisher requires approximately $2^{85.6}$ encryptions. The parameters and distinguishers used in this estimation, as well as those used in our 11-round attack, are summarized in Appendix D and E, respectively and detailed explanations are omitted here for brevity.

6.2 Our Key-Recovery Attack on 12-round FLY

We describe our key-recovery attack on 12-round FLY as the following steps:

1. Assign all 2^{64} values for $P[0, 1, \dots, 7]$. Then, encrypt these plaintexts and store the corresponding ciphertexts in a hash table.
2. Guess the 64-bit subkey RK_0 .
3. Guess the 16-bit subkey $RK_1[2, 5]$, i.e., $K_{\text{in}} = RK_0 \times RK_1[2, 5]$.
4. For each $k \in K_{\text{in}}$, retrieve $4 \times 2^{56.77}$ pairs $(P, \tilde{P})$ from the hash table satisfying

$$E_{\kappa_{\text{in}}}^{\text{in}}(P) \oplus E_{\kappa_{\text{in}}}^{\text{in}}(\tilde{P}) = \Delta S_2, \quad E_K(P) \oplus E_K(\tilde{P}) \in D_{\text{out}}$$

and construct the set

$$\mathcal{C} := \{(C, \tilde{C}) : C = E_K(P), \tilde{C} = E_K(\tilde{P})\},$$

where $\|\mathcal{C}\| = 2^{15-16} = 2^{-1}$.

5. Initialize a 2^{92} key counter $\text{cnt}[\cdot]$.
6. For each ciphertext pair $(C, \tilde{C}) \in \mathcal{C}$, compute $(x_{\mathbf{R}}^{12}, \tilde{x}_{\mathbf{R}}^{12}) = (\mathbf{R}^{-1}(C), \mathbf{R}^{-1}(\tilde{C}))$.

7. Using the key schedule and the RK_{12} already known from the initial guess of RK_0 .
8. Compute

$$x_{\mathbf{R}}^{12} = \mathbf{S}^{-1} \left(x_{\mathbf{S}}^{12} \oplus \overline{RK_{12}^{\oplus}} \right),$$

$$\tilde{x}_{\mathbf{R}}^{12} = \mathbf{S}^{-1} \left(\tilde{x}_{\mathbf{S}}^{12} \oplus \overline{RK_{12}^{\oplus}} \right),$$

9. Using the key schedule and the 2 bits already known from the initial guess of $RK_1[2, 5]$, guess the remaining 6 bits of each byte $\overline{RK_{11}^{\oplus}}[1]$ and $\overline{RK_{11}^{\oplus}}[5]$. Then apply the inverse S-boxes and verify whether the resulting differences match the expected values ∇
10. If all conditions are satisfied, increment the corresponding 92-bit key counter:

$$\text{cnt} \left[\underbrace{RK_0[0, \dots, 7] \parallel RK_1[2, 5]}_{80\text{-bit}} \parallel \underbrace{\overline{RK_{11}^{\oplus}}[1, 5]}_{12\text{-bit}} \right]$$

For the detailed values of $\Delta \mathbf{S}^r$, $\Delta \mathbf{R}^r$, and $\Delta \mathbf{K}^r$, see Table F19 in Appendix F.

6.2.1 Complexity Analysis

The parameters used in the attack and the values obtained at each step of successive guessing and filtering are summarized in Table 4 and Table 5, respectively.

Accordingly, the overall complexities and the success probability are given by Eqs. 1–4 as follows:

Table 4: Parameter settings for the 11-round PIPO attack

Symbol	Value
μ	4
p	56.77
N_{Δ, r_f}	10
M_{∇, r_b}	10
$\Lambda(\mu, p, d_{\text{in}})$	2^{64}
d_{in}	64
d_{out}	48
k_{in}	80
k_{out}	12
c_e	$2^{6.46}$
K_m	128
a	58

Table 5: Successive steps of guessing and filtering in key counting for the 12-round FLY attack

i	$g(i)$	$f(i)$	$\sum_{j=0}^i (g(j) - f(j))$
0	0	0	0
1	0	0	0
2	0	0	0
3	0	0	0
4	0	0	0
5	0	0	0
6	0	0	0
7	0	0	0
8	6	0	6
9	6	4	8

Then the overall time complexity becomes:

$$\mathcal{T} \approx 4 \times 2^{56.77+1+80-16} \cdot \{1 + 2^{-6.46} \cdot (10 + 1 + \dots + 2^8)\} + 2^{64} + 2^{128-58} = 2^{126.045} \quad (\text{encs}).$$

Accordingly, the memory complexity is given by

$$\mathcal{M} = 8 \cdot 2^{64} + 8 \cdot 4 \cdot 2^{56.77-16} + 2^{80+12} \approx 2^{92} \quad (\text{bytes}).$$

In terms of data requirements, the attack needs

$$\mathcal{D} = 4 \times 2^{56.77+1} = 2^{59.77} \quad (\text{CP}).$$

Finally, the success probability of the attack is

$$\mathbb{P}_S = 0.9 \quad (90\%).$$

6.3 Key-Recovery Attack without Full Codebook

The imposed condition technique is a general method for achieving time–data–memory trade-offs and can be applied independently of the internal components of the cipher. However, the associated computation is not tight. If the key-recovery rounds before the distinguisher are short, the S-box structure can be leveraged to tighten the trade-off.

Let $\mathcal{S}_i$ denote a structure in which the i -th column of the plaintexts satisfies $\Delta \mathbf{S}_1[i] = 0$, while all other positions take on all possible values. We consider the case where a pair generated from $\mathcal{S}_i$ satisfies the condition $\Delta \mathbf{S}_2$. The corresponding probability is given by:

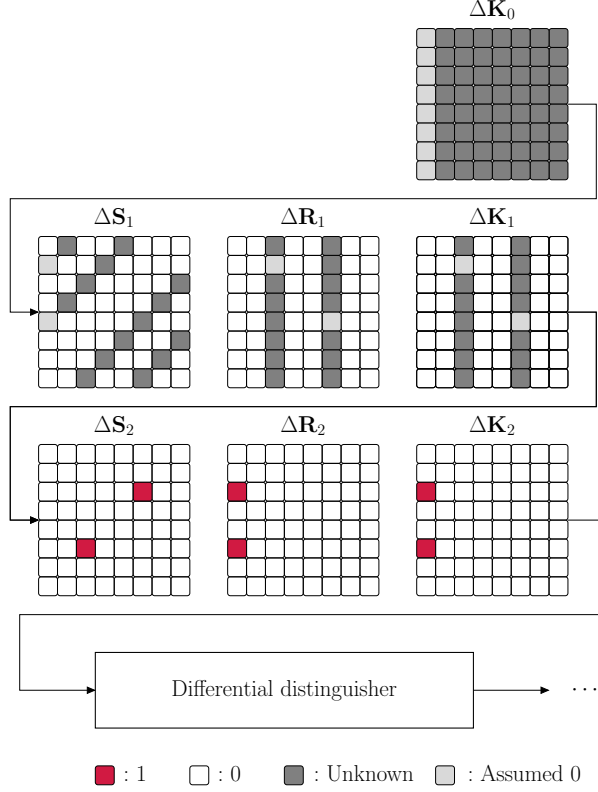


Fig. 10: Time-data-memory trade-off attack on 12-round FLY.

$$\mathbb{P}_{P, \kappa_{\text{in}}} \left[\tilde{P} \in \mathcal{S}_i \mid \Delta \mathbf{S}_1[i] = 0 \wedge E_{\kappa_{\text{in}}}^{\text{in}}(P) \oplus E_{\kappa_{\text{in}}}^{\text{in}}(\tilde{P}) = \Delta \mathbf{S}_2 \right]. \quad (5)$$

For example, as illustrated in Fig. 10, when the 0-th column of the plaintext satisfies $\Delta \mathbf{S}_1[0] = 0$ and all other positions are filled with all possible values, the corresponding probability becomes:

$$\mathbb{P}[00100000 \xrightarrow{S^{-1}} *0*****] \cdot \mathbb{P}[00000100 \xrightarrow{S^{-1}} ****0***] = 2^{-0.678} \cdot 2^{-0.678} = 2^{-1.36}.$$

The value of Eq. 5 depends on the index i , and the resulting probabilities are summarized in Table 6. As in Section 5.2.2, where the imposed condition was applied, we apply it in the same manner to our differential key-recovery framework for 12-round FLY, and obtain the following complexity estimates:

$$\begin{aligned} \mathcal{T} \approx & 4 \times 2^{56.77+1+80-16+1.36} \cdot \{1 + 2^{-6.46} \cdot (10 + 1 + \dots + 2^8)\} \\ & + 4 \times 2^{56.77+1.36} + 2^{128-58} = 2^{127.405} \quad (\text{encs}). \end{aligned}$$

The corresponding memory complexity is:

$$\mathcal{M} = 8 \cdot 2^{56.77+1.36} + 8 \cdot 4 \cdot 2^{56.77-16} + 2^{80+12} \approx 2^{92} \quad (\text{bytes}).$$

The data complexity is given by:

$$\mathcal{D} = 4 \times 2^{56.77+1+1.36} = 2^{61.13} \quad (\text{CP}).$$

The success probability of the attack remains unchanged at $\mathbb{P}_S = 0.9$ (90%).

Table 6: Expected probabilities of Eq. 5 for each column index i .

i	Input	Output	Probability
0	00100000 00000100	*0***** ****0***	$\mathbb{P}[00100000 \xrightarrow{S^{-1}} *0*****] \cdot \mathbb{P}[00000100 \xrightarrow{S^{-1}} ****0***] = 2^{-0.678} \cdot 2^{-0.678} = 2^{-1.36}$
1	00100000 00000100	**0***** *****0**	$\mathbb{P}[00100000 \xrightarrow{S^{-1}} **0*****] \cdot \mathbb{P}[00000100 \xrightarrow{S^{-1}} *****0**] = 2^{-1} \cdot 2^{-1.193} = 2^{-2.193}$
2	00100000 00000100	***0**** *****0*	$\mathbb{P}[00100000 \xrightarrow{S^{-1}} ***0****] \cdot \mathbb{P}[00000100 \xrightarrow{S^{-1}} *****0*] = 2^{-1} \cdot 2^{-1} = 2^{-2}$
3	00100000 00000100	****0*** *****0	$\mathbb{P}[00100000 \xrightarrow{S^{-1}} ****0***] \cdot \mathbb{P}[00000100 \xrightarrow{S^{-1}} *****0] = 2^{-2} \cdot 2^{-2} = 2^{-4}$
4	00100000 00000100	*****0** 0*****	$\mathbb{P}[00100000 \xrightarrow{S^{-1}} *****0**] \cdot \mathbb{P}[00000100 \xrightarrow{S^{-1}} 0*****] = 2^{-0.678} \cdot 2^{-0.678} = 2^{-1.36}$
5	00100000 00000100	*****0* *0*****	$\mathbb{P}[00100000 \xrightarrow{S^{-1}} *****0*] \cdot \mathbb{P}[00000100 \xrightarrow{S^{-1}} *0*****] = 2^{-1} \cdot 2^{-1.193} = 2^{-2.193}$
6	00100000 00000100	*****0 **0*****	$\mathbb{P}[00100000 \xrightarrow{S^{-1}} *****0] \cdot \mathbb{P}[00000100 \xrightarrow{S^{-1}} **0*****] = 2^{-1} \cdot 2^{-1} = 2^{-2}$
7	00100000 00000100	0***** ****0***	$\mathbb{P}[00100000 \xrightarrow{S^{-1}} 0*****] \cdot \mathbb{P}[00000100 \xrightarrow{S^{-1}} ****0***] = 2^{-2} \cdot 2^{-2} = 2^{-4}$

7 Conclusion

We have proposed improved differential attacks on PIPO and FLY, addressing the gap in the existing analysis of these lightweight block ciphers. For PIPO-64-128, we achieved a significant time complexity reduction of $2^{31.7}$ by constructing optimal differential trails that are key-recovery-friendly. For PIPO-64-256, we extended the previous analysis and presented a 13-round key-recovery attack, advancing the attack boundary by two rounds. In addition, we conducted the first differential attack on FLY, which extended the attack to 12 rounds, one round beyond the previously known distinguishers.

Our attacks are based on a generalized differential attack framework, which is applicable to ciphers with similar structures. This methodology provides an effective strategy for improving the efficiency of key-recovery attacks on lightweight block ciphers.

Appendix A S-box of FLY and PIPO

Table A1: S-box lookup table of PIPO (hexadecimal)

$\begin{smallmatrix} y \\ x \end{smallmatrix}$	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	5E	F9	FC	00	3F	85	BA	5B	18	37	B2	C6	71	C3	74	9D
1	A7	94	0D	E1	CA	68	53	2E	49	62	EB	97	A4	0E	2D	D0
2	16	25	AC	48	63	D1	EA	8F	F7	40	45	B1	9E	34	1B	F2
3	B9	86	03	7F	D8	7A	DD	3C	E0	CB	52	26	15	AF	8C	69
4	C2	75	70	1C	33	99	B6	C7	04	3B	BE	5A	FD	5F	F8	81
5	93	A0	29	4D	66	D4	EF	0A	E5	CE	57	A3	90	2A	09	6C
6	22	11	88	E4	CF	6D	56	AB	7B	DC	D9	BD	82	38	07	7E
7	B5	9A	1F	F3	44	F6	41	30	4C	67	EE	12	21	8B	A8	D5
8	55	6E	E7	0B	28	92	A1	CC	2B	08	91	ED	D6	64	4F	A2
9	BC	83	06	FA	5D	FF	58	39	72	C5	C0	B4	9B	31	1E	77
A	01	3E	BB	DF	78	DA	7D	84	50	6B	E2	8E	AD	17	24	C9
B	AE	8D	14	E8	D3	61	4A	27	47	F0	F5	19	36	9C	B3	42
C	1D	32	B7	43	F4	46	F1	98	EC	D7	4E	AA	89	23	10	65
D	8A	A9	20	54	6F	CD	E6	13	DB	7C	79	05	3A	80	BF	DE
E	E9	D2	4B	2F	0C	A6	95	60	0F	2C	A5	51	6A	C8	E3	96
F	B0	9F	1A	76	C1	73	C4	35	FE	59	5C	B8	87	3D	02	FB

Table A2: S-box lookup table of FLY (hexadecimal)

$\begin{smallmatrix} y \\ x \end{smallmatrix}$	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	00	9B	C2	15	5D	84	4C	D1	67	38	EF	B0	7E	2B	F6	A3
1	B9	AA	36	78	2F	6E	E3	F7	12	5C	9A	D4	89	CD	01	45
2	2C	63	44	DE	02	96	39	70	BA	E4	18	57	A1	F5	8B	CE
3	51	87	ED	FF	B5	A8	CA	1B	DF	90	6C	32	46	03	7D	29
4	D5	F2	20	5B	CC	31	04	BD	A6	41	8E	79	EA	9F	68	1C
5	48	E6	69	8A	13	77	9E	AF	F3	05	CB	2D	B4	D0	37	52
6	C4	3E	93	AC	40	E9	22	56	7B	8D	F1	06	17	62	BF	DA
7	1D	7F	07	B1	DB	FA	65	88	2E	C9	A5	43	58	3C	E0	94
8	76	21	AB	FD	6A	3F	B7	E2	DD	4F	53	8C	C0	19	95	08
9	83	C5	4E	09	14	50	D8	9C	F4	EE	27	61	3B	7A	A2	B6
A	FE	A9	81	C6	E8	BC	1F	5A	35	72	99	0A	D3	47	24	6D
B	0B	4D	75	23	97	D2	60	34	C8	16	A0	BB	FC	E1	5E	8F
C	E7	98	1A	64	AE	4B	71	85	0C	B3	3D	CF	55	28	D9	F0
D	B2	DC	5F	30	F9	0D	26	C3	91	A7	74	1E	82	66	4A	EB
E	6F	10	B8	D7	86	73	FB	0E	59	2A	42	E5	9D	A4	33	C7
F	3A	54	EC	92	C1	25	AD	49	80	6B	D6	F8	0F	BE	7C	11

Appendix B Details of the Key-Recovery Attack on 9-Round PIPO-64-128

Due to space limitations, only a subset of the characteristics is shown. For the complete list of trails used in our analysis, please refer to our code repository.

Table B3: 6-round differential characteristic of PIPO (T_0)

Round	Type	Value	Round	Type	Value
R1	SBOXIN	41 00 00 00 00 00 00 00	R2	SBOXIN	00 00 00 00 00 00 00 40
	SBOXOUT	40 00 00 00 00 00 00 00		SBOXOUT	00 00 00 00 00 00 00 40
	DIFFOUT	00 00 00 00 00 00 00 40		DIFFOUT	00 00 00 00 08 00 40 00
	PROB	2−5.4150		PROB	2−6.0000
R3	SBOXIN	00 00 00 00 08 00 40 00	R4	SBOXIN	00 00 84 48 81 00 00 20
	SBOXOUT	00 00 00 00 E1 00 8C 00		SBOXOUT	00 00 14 22 10 00 00 48
	DIFFOUT	00 00 84 48 81 00 00 20		DIFFOUT	00 00 00 00 1A 00 74 00
	PROB	2−9.0000		PROB	2−21.4150
R5	SBOXIN	00 00 00 00 1A 00 74 00	R6	SBOXIN	00 28 00 00 00 00 00 00
	SBOXOUT	00 00 00 00 08 00 20 00		SBOXOUT	00 05 00 00 00 00 00 00
	DIFFOUT	00 28 00 00 00 00 00 00		DIFFOUT	00 01 00 00 00 04 00 00
	PROB	2−13.0000		PROB	2−4.0000
Total Probability: 2−58.8301					

Table B4: 6-round differential characteristic of PIPO (T_1)

Round	Type	Value	Round	Type	Value
R1	SBOXIN	41 00 00 00 00 00 00 00	R2	SBOXIN	00 00 00 00 00 00 80 40
	SBOXOUT	C0 00 00 00 00 00 00 00		SBOXOUT	00 00 00 00 00 00 85 E8
	DIFFOUT	00 00 00 00 00 00 80 40		DIFFOUT	00 00 24 00 88 80 41 00
	PROB	2−5.4150		PROB	2−11.0000
R3	SBOXIN	00 00 24 00 88 80 41 00	R4	SBOXIN	84 0C 00 00 00 63 00 00
	SBOXOUT	00 00 A0 00 0E 05 40 00		SBOXOUT	84 08 00 00 00 42 00 00
	DIFFOUT	84 0C 00 00 00 63 00 00		DIFFOUT	00 00 00 00 44 00 8A 00
	PROB	2−24.4150		PROB	2−16.4150
R5	SBOXIN	00 00 00 00 44 00 8A 00	R6	SBOXIN	00 28 00 00 00 00 00 00
	SBOXOUT	00 00 00 00 08 00 20 00		SBOXOUT	00 05 00 00 00 00 00 00
	DIFFOUT	00 28 00 00 00 00 00 00		DIFFOUT	00 01 00 00 00 04 00 00
	PROB	2−13.0000		PROB	2−4.0000
Total Probability: 2−74.2451					

Table B5: 6-round differential characteristic of PIPO (T_2)

Round	Type	Value	Round	Type	Value
R1	SBOXIN	41 00 00 00 00 00 00 00	R2	SBOXIN	00 00 00 00 00 00 80 40
	SBOXOUT	00 00 00 00 00 00 00 00		SBOXOUT	00 00 00 00 00 00 F0 12
	DIFFOUT	00 00 00 00 00 00 80 40		DIFFOUT	12 30 00 00 80 40 00 00
	PROB	2−5.4150		PROB	2−10.0000
R3	SBOXIN	12 30 00 00 80 40 00 00	R4	SBOXIN	25 00 9A 00 25 00 02 00
	SBOXOUT	15 22 00 00 85 2A 00 00		SBOXOUT	90 00 05 00 90 00 05 00
	DIFFOUT	25 00 9A 00 25 00 02 00		DIFFOUT	00 00 95 00 00 00 95 00
	PROB	2−21.0000		PROB	2−22.0000
R5	SBOXIN	00 00 95 00 00 00 95 00	R6	SBOXIN	00 60 00 00 00 00 00 00
	SBOXOUT	00 00 40 00 00 00 20 00		SBOXOUT	00 05 00 00 00 00 00 00
	DIFFOUT	00 60 00 00 00 00 00 00		DIFFOUT	00 01 00 00 00 04 00 00
	PROB	2−12.0000		PROB	2−4.0000
Total Probability: 2−74.4150					

Table B6: Difference Values of ΔK^i , ΔS^i , and ΔR^i used in the key-recovery attack on 9-Round PIPO-64-128.

	0-th col.	1-st col.	2-nd col.	3-rd col.	4-th col.	5-th col.	6-th col.	7-th col.
ΔK^0	*****	*****	00000000	00000000	00000000	00000000	00000000	00000000
ΔS^1	00000001	01000000	00000000	00000000	00000000	00000000	00000000	00000000
ΔR^1	01000001	00000000	00000000	00000000	00000000	00000000	00000000	00000000
ΔK^1	01000001	00000000	00000000	00000000	00000000	00000000	00000000	00000000
ΔK^7	00000000	00000101	00000000	00000000	00000000	00000*00	00000000	00000000
ΔS^8	00000000	**1*****	00000000	00000000	00000000	*1*****	00000000	00000000
ΔR^8	0**00000	00000*0*	0000*0*0	*00*0000	01100000	00000*0*	0000*0*0	*00*0000
ΔK^8	0**00000	00000*0*	0000*0*0	*00*0000	01100000	00000*0*	0000*0*0	*00*0000
ΔS^9	*****	*****	*****	*****	*****	*****	*****	*****
ΔR^9	*****	*****	*****	*****	*****	*****	*****	*****
ΔK^9	*****	*****	*****	*****	*****	*****	*****	*****

Appendix C Details of the Key-Recovery Attack on 13-Round PIPO-64-256

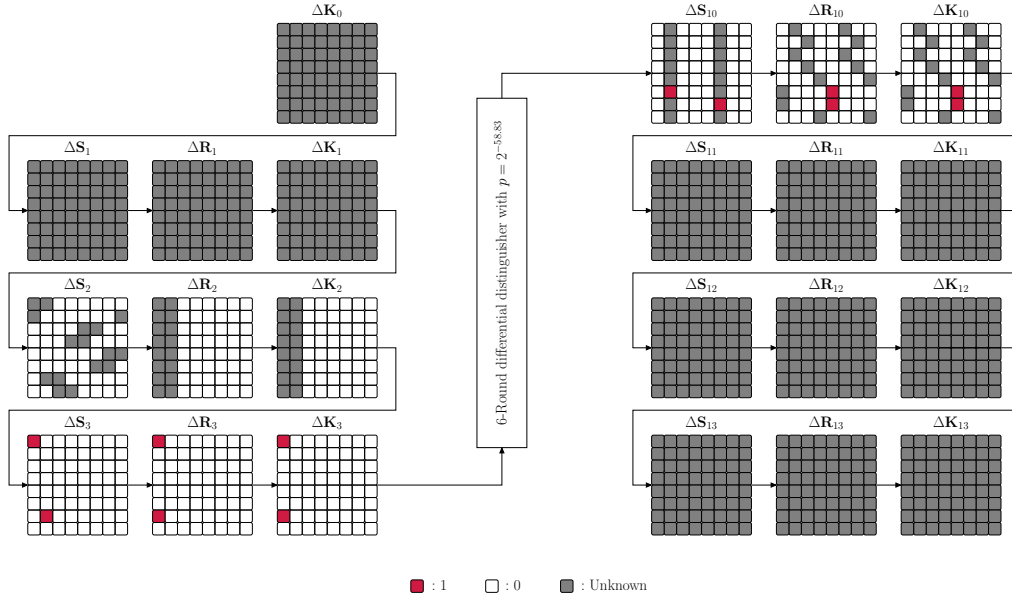


Fig. C1: 13-round key-recovery attack on PIPO-64-256

Table C7: Difference Values of ΔK^i , ΔS^i , and ΔR^i used in the key-recovery attack on 13-Round PIPO-64-256.

	0-th col.	1-st col.	2-nd col.	3-rd col.	4-th col.	5-th col.	6-th col.	7-th col.
ΔK^0	*****	*****	*****	*****	*****	*****	*****	*****
ΔS^1	*****	*****	*****	*****	*****	*****	*****	*****
ΔR^1	*****	*****	*****	*****	*****	*****	*****	*****
ΔK^1	*****	*****	*****	*****	*****	*****	*****	*****
ΔS^2	000000**	0*00000*	**000000	*000*000	0000**00	00*00*00	00**0000	000*00*0
ΔR^2	*****	*****	00000000	00000000	00000000	00000000	00000000	00000000
ΔK^2	*****	*****	00000000	00000000	00000000	00000000	00000000	00000000
ΔS^3	00000001	01000000	00000000	00000000	00000000	00000000	00000000	00000000
ΔR^3	01000001	00000000	00000000	00000000	00000000	00000000	00000000	00000000
ΔK^3	01000001	00000000	00000000	00000000	00000000	00000000	00000000	00000000
ΔK^9	00000000	00000101	00000000	00000000	00000000	00000*00	00000000	00000000
ΔS^{10}	00000000	**1*****	00000000	00000000	00000000	*1*****	00000000	00000000
ΔR^{10}	0**00000	00000*0*	0000*0*0	*00*0000	01100000	00000*0*	0000*0*0	*00*0000
ΔK^{10}	0**00000	00000*0*	0000*0*0	*00*0000	01100000	00000*0*	0000*0*0	*00*0000
ΔS^{11}	*****	*****	*****	*****	*****	*****	*****	*****
ΔR^{11}	*****	*****	*****	*****	*****	*****	*****	*****
ΔK^{11}	*****	*****	*****	*****	*****	*****	*****	*****
ΔS^{12}	*****	*****	*****	*****	*****	*****	*****	*****
ΔR^{12}	*****	*****	*****	*****	*****	*****	*****	*****
ΔK^{12}	*****	*****	*****	*****	*****	*****	*****	*****
ΔS^{13}	*****	*****	*****	*****	*****	*****	*****	*****
ΔR^{13}	*****	*****	*****	*****	*****	*****	*****	*****
ΔK^{13}	*****	*****	*****	*****	*****	*****	*****	*****

Table C8: Parameters and successive guessing and filtering steps for the 13-round PIPO-64-256 attack.

(a) Parameters		(b) Successive steps			
Symbol	Value	i	$g(i)$	$f(i)$	$\sum_{j=0}^i (g(j) - f(j))$
μ	6	0	0	0	0
p	58.83	$\vdots$	$\vdots$	$\vdots$	0
N_{Δ, r_f}	18	15	0	0	0
M_{∇, r_b}	26	16	8	0	8
$\Lambda(\mu, p, d_{\text{in}})$	2^{64}	17	8	8	8
d_{in}	64	18	8	6	10
d_{out}	64	19	8	6	12
k_{in}	144	20	8	6	14
k_{out}	76	21	8	6	16
c_e	$2^{6.17}$	22	8	6	18
K_m	256	23	8	6	20
a	38	24	6	6	20
		25	6	7	19

Appendix D Details of the Key-Recovery Attack based on the distinguisher in [11]

Using the program provided by the authors of [11], we obtained 20 differential distinguisher with probability $2^{-55.77}$. We then applied our key-recovery framework to these trails and selected the one with the lowest time complexity as a baseline for comparison.

We calculate the overall key-recovery complexity as follows:

Table D9: Parameters and successive guessing and filtering steps for the 11-round FLY attack based on [11].

(a) Parameters		(b) Successive steps			
Symbol	Value	i	$g(i)$	$f(i)$	$\sum_{j=0}^i (g(j) - f(j))$
μ	4	0	8	0	8
p	56.77	1	8	8	8
N_{Δ, r_f}	2	2	8	8	8
M_{∇, r_b}	10	3	8	6	10
$\Lambda(\mu, p, d_{\text{in}})$	$4 \times 2^{56.77+1}$	4	8	6	12
d_{in}	16	5	8	6	14
d_{out}	48	6	8	6	16
k_{in}	16	7	8	0	24
k_{out}	76	8	6	0	30
c_e	$2^{6.46}$	9	6	4	32
K_m	128				
a	58				

$$\begin{aligned} \mathcal{T} \approx & 4 \times 2^{56.77+1+16-16} \cdot \{1 + 2^{-6.46} \cdot (2 + 2^8 + \dots + 2^{32})\} \\ & + 4 \times 2^{56.77+1} + 2^{128-58} = 2^{85.636} \quad (\text{encs}). \end{aligned}$$

$$\mathcal{M} = 8 \cdot 2^{16} + 8 \cdot 2^0 + 2^{16+76} \approx 2^{92} \quad (\text{bytes}).$$

$$\mathcal{D} = 4 \times 2^{56.77+1} = 2^{59.77} \quad (\text{CP}).$$

$$\mathbb{P}_S = 0.9 \quad (90\%).$$

Table D10: Difference Values of ΔK^i , ΔS^i , and ΔR^i used in the key-recovery attack on 11-Round FLY based on [11].

	0-th col.	1-st col.	2-nd col.	3-rd col.	4-th col.	5-th col.	6-th col.	7-th col.
ΔK^0	00000000	*****	*****	00000000	00000000	*****	00000000	00000000
ΔS^1	00000000	10000000	01000000	00000000	00000000	00001000	00000000	00000000
ΔR^1	00000000	11001000	00000000	00000000	00000000	00000000	00000000	00000000
ΔK^1	00000000	11001000	00000000	00000000	00000000	00000000	00000000	00000000
ΔK^9	00000000	0*000*00	01000100	00000000	00000000	0*000*00	01000100	00000000
ΔS^{11}	00000000	*****	00000000	00000000	00000000	*****	00000000	00000000
ΔR^{11}	0*000*00	*000*000	000*000*	00*000*0	0*000*00	*000*000	000*000*	00*000*0
ΔK^{11}	0*000*00	*000*000	000*000*	00*000*0	0*000*00	*000*000	000*000*	00*000*0
ΔS^{12}	*****	*****	*****	*****	*****	*****	*****	*****
ΔR^{12}	*****	*****	*****	*****	*****	*****	*****	*****
ΔK^{12}	*****	*****	*****	*****	*****	*****	*****	*****

Appendix E Details of the Our Key-Recovery Attack on 11-Round FLY

Table E11: 8-round differential characteristic of FLY (T_0)

Round	Type	Value	Round	Type	Value
R1	SBOXIN	24 00 00 00 00 00 00 00	R2	SBOXIN	00 00 02 00 00 00 00 00
	SBOXOUT	02 00 00 00 00 00 00 00		SBOXOUT	00 00 88 00 00 00 00 00
	DIFFOUT	00 00 02 00 00 00 00 00		DIFFOUT	00 00 80 00 00 00 08 00
	PROB	2-4.0000		PROB	2-4.0000
R3	SBOXIN	00 00 80 00 00 00 08 00	R4	SBOXIN	00 44 88 00 00 00 88 00
	SBOXOUT	00 00 C8 00 00 00 8C 00		SBOXOUT	00 88 44 00 00 00 44 00
	DIFFOUT	00 44 88 00 00 00 88 00		DIFFOUT	00 C4 00 00 00 4C 00 00
	PROB	2-14.0000		PROB	2-12.0000
R5	SBOXIN	00 C4 00 00 00 4C 00 00	R6	SBOXIN	00 00 00 00 00 00 11 00
	SBOXOUT	00 10 00 00 00 01 00 00		SBOXOUT	00 00 00 00 00 00 22 00
	DIFFOUT	00 00 00 00 00 00 11 00		DIFFOUT	02 00 00 00 20 00 00 00
	PROB	2-8.0000		PROB	2-4.0000
R7	SBOXIN	02 00 00 00 20 00 00 00	R8	SBOXIN	88 00 00 00 88 00 00 00
	SBOXOUT	88 00 00 00 88 00 00 00		SBOXOUT	11 00 00 00 11 00 00 00
	DIFFOUT	88 00 00 00 88 00 00 00		DIFFOUT	00 11 00 00 00 11 00 00
	PROB	2-8.0000		PROB	2-8.0000
Total Probability: 2-62.0000					

Due to space limitations, only a subset of the characteristics is shown. For the complete list of trails used in our analysis, please refer to our code repository.

Table E12: 8-round differential characteristic of FLY (T_1)

Round	Type	Value	Round	Type	Value
R1	SBOXIN	24 00 00 00 00 00 00 00	R2	SBOXIN	00 00 02 00 00 00 00 00
	SBOXOUT	02 00 00 00 00 00 00 00		SBOXOUT	00 00 88 00 00 00 00 00
	DIFFOUT	00 00 02 00 00 00 00 00		DIFFOUT	00 00 80 00 00 00 08 00
	PROB	2−4.0000		PROB	2−4.0000
R3	SBOXIN	00 00 80 00 00 00 08 00	R4	SBOXIN	00 00 88 00 00 44 88 00
	SBOXOUT	00 00 8C 00 00 00 C8 00		SBOXOUT	00 00 44 00 00 88 44 00
	DIFFOUT	00 00 88 00 00 44 88 00		DIFFOUT	00 4C 00 00 00 C4 00 00
	PROB	2−14.0000		PROB	2−12.0000
R5	SBOXIN	00 4C 00 00 00 C4 00 00	R6	SBOXIN	00 00 11 00 00 00 00 00
	SBOXOUT	00 01 00 00 00 10 00 00		SBOXOUT	00 00 22 00 00 00 00 00
	DIFFOUT	00 00 11 00 00 00 00 00		DIFFOUT	20 00 00 00 02 00 00 00
	PROB	2−8.0000		PROB	2−4.0000
R7	SBOXIN	20 00 00 00 02 00 00 00	R8	SBOXIN	88 00 00 00 88 00 00 00
	SBOXOUT	88 00 00 00 88 00 00 00		SBOXOUT	11 00 00 00 11 00 00 00
	DIFFOUT	88 00 00 00 88 00 00 00		DIFFOUT	00 11 00 00 00 11 00 00
	PROB	2−8.0000		PROB	2−8.0000
Total Probability: 2−62.0000					

Table E13: 8-round differential characteristic of FLY (T_2)

Round	Type	Value	Round	Type	Value
R1	SBOXIN	24 00 00 00 00 00 00 00	R2	SBOXIN	00 00 00 00 08 00 00 00
	SBOXOUT	08 00 00 00 00 00 00 00		SBOXOUT	00 00 00 00 22 00 00 00
	DIFFOUT	00 00 00 00 08 00 00 00		DIFFOUT	00 00 20 00 00 00 02 00
	PROB	2-4.0000		PROB	2-6.0000
R3	SBOXIN	00 00 20 00 00 00 02 00	R4	SBOXIN	00 00 88 00 00 00 88 00
	SBOXOUT	00 00 88 00 00 00 88 00		SBOXOUT	00 00 44 00 00 00 44 00
	DIFFOUT	00 00 88 00 00 00 88 00		DIFFOUT	00 44 00 00 00 44 00 00
	PROB	2-8.0000		PROB	2-8.0000
R5	SBOXIN	00 44 00 00 00 44 00 00	R6	SBOXIN	00 88 00 00 00 88 00 00
	SBOXOUT	00 88 00 00 00 88 00 00		SBOXOUT	00 44 00 00 00 44 00 00
	DIFFOUT	00 88 00 00 00 88 00 00		DIFFOUT	44 00 00 00 44 00 00 00
	PROB	2-8.0000		PROB	2-8.0000
R7	SBOXIN	44 00 00 00 44 00 00 00	R8	SBOXIN	88 00 00 00 88 00 00 00
	SBOXOUT	88 00 00 00 88 00 00 00		SBOXOUT	11 00 00 00 11 00 00 00
	DIFFOUT	88 00 00 00 88 00 00 00		DIFFOUT	00 11 00 00 00 11 00 00
	PROB	2-8.0000		PROB	2-8.0000
Total Probability: 2-58.0000					

Table E14: 8-round differential characteristic of FLY (T_3)

Round	Type	Value	Round	Type	Value
R1	SBOXIN	24 00 00 00 00 00 00 00	R2	SBOXIN	00 00 00 00 08 00 00 00
	SBOXOUT	08 00 00 00 00 00 00 00		SBOXOUT	00 00 00 00 22 00 00 00
	DIFFOUT	00 00 00 00 08 00 00 00		DIFFOUT	00 00 20 00 00 00 02 00
	PROB	2-4.0000		PROB	2-6.0000
R3	SBOXIN	00 00 20 00 00 00 02 00	R4	SBOXIN	00 00 88 00 00 00 88 00
	SBOXOUT	00 00 88 00 00 00 88 00		SBOXOUT	00 00 11 00 00 00 11 00
	DIFFOUT	00 00 88 00 00 00 88 00		DIFFOUT	00 00 00 11 00 00 00 11
	PROB	2-8.0000		PROB	2-8.0000
R5	SBOXIN	00 00 00 11 00 00 00 11	R6	SBOXIN	00 22 00 00 00 22 00 00
	SBOXOUT	00 00 00 22 00 00 00 22		SBOXOUT	00 44 00 00 00 44 00 00
	DIFFOUT	00 22 00 00 00 22 00 00		DIFFOUT	44 00 00 00 44 00 00 00
	PROB	2-8.0000		PROB	2-8.0000
R7	SBOXIN	44 00 00 00 44 00 00 00	R8	SBOXIN	88 00 00 00 88 00 00 00
	SBOXOUT	88 00 00 00 88 00 00 00		SBOXOUT	11 00 00 00 11 00 00 00
	DIFFOUT	88 00 00 00 88 00 00 00		DIFFOUT	00 11 00 00 00 11 00 00
	PROB	2-8.0000		PROB	2-8.0000
Total Probability: 2-58.0000					

Table E15: 8-round differential characteristic of FLY (T_4)

Round	Type	Value	Round	Type	Value
R1	SBOXIN	24 00 00 00 00 00 00 00	R2	SBOXIN	00 00 00 00 08 00 00 00
	SBOXOUT	08 00 00 00 00 00 00 00		SBOXOUT	00 00 00 00 22 00 00 00
	DIFFOUT	00 00 00 00 08 00 00 00		DIFFOUT	00 00 20 00 00 00 02 00
	PROB	2-4.0000		PROB	2-6.0000
R3	SBOXIN	00 00 20 00 00 00 02 00	R4	SBOXIN	00 44 88 00 00 00 88 00
	SBOXOUT	00 00 C8 00 00 00 8C 00		SBOXOUT	00 88 44 00 00 00 44 00
	DIFFOUT	00 44 88 00 00 00 88 00		DIFFOUT	00 C4 00 00 00 4C 00 00
	PROB	2-12.0000		PROB	2-12.0000
R5	SBOXIN	00 C4 00 00 00 4C 00 00	R6	SBOXIN	00 00 00 00 00 00 11 00
	SBOXOUT	00 10 00 00 00 01 00 00		SBOXOUT	00 00 00 00 00 00 22 00
	DIFFOUT	00 00 00 00 00 00 11 00		DIFFOUT	02 00 00 00 20 00 00 00
	PROB	2-8.0000		PROB	2-4.0000
R7	SBOXIN	02 00 00 00 20 00 00 00	R8	SBOXIN	88 00 00 00 88 00 00 00
	SBOXOUT	88 00 00 00 88 00 00 00		SBOXOUT	11 00 00 00 11 00 00 00
	DIFFOUT	88 00 00 00 88 00 00 00		DIFFOUT	00 11 00 00 00 11 00 00
	PROB	2-8.0000		PROB	2-8.0000
Total Probability: 2-62.0000					

Table E16: 8-round differential characteristic of FLY (T_5)

Round	Type	Value	Round	Type	Value
R1	SBOXIN	24 00 00 00 00 00 00 00	R2	SBOXIN	00 00 00 00 08 00 00 00
	SBOXOUT	08 00 00 00 00 00 00 00		SBOXOUT	00 00 00 00 22 00 00 00
	DIFFOUT	00 00 00 00 08 00 00 00		DIFFOUT	00 00 20 00 00 00 02 00
	PROB	2−4.0000		PROB	2−6.0000
R3	SBOXIN	00 00 20 00 00 00 02 00	R4	SBOXIN	00 00 88 00 00 44 88 00
	SBOXOUT	00 00 8C 00 00 00 C8 00		SBOXOUT	00 00 44 00 00 88 44 00
	DIFFOUT	00 00 88 00 00 44 88 00		DIFFOUT	00 4C 00 00 00 C4 00 00
	PROB	2−12.0000		PROB	2−12.0000
R5	SBOXIN	00 4C 00 00 00 C4 00 00	R6	SBOXIN	00 00 11 00 00 00 00 00
	SBOXOUT	00 01 00 00 00 10 00 00		SBOXOUT	00 00 22 00 00 00 00 00
	DIFFOUT	00 00 11 00 00 00 00 00		DIFFOUT	20 00 00 00 02 00 00 00
	PROB	2−8.0000		PROB	2−4.0000
R7	SBOXIN	20 00 00 00 02 00 00 00	R8	SBOXIN	88 00 00 00 88 00 00 00
	SBOXOUT	88 00 00 00 88 00 00 00		SBOXOUT	11 00 00 00 11 00 00 00
	DIFFOUT	88 00 00 00 88 00 00 00		DIFFOUT	00 11 00 00 00 11 00 00
	PROB	2−8.0000		PROB	2−8.0000
Total Probability: 2−62.0000					

Table E17: Difference Values of ΔK^i , ΔS^i , and ΔR^i used in the key-recovery attack on 11-Round FLY.

	0-th col.	1-st col.	2-nd col.	3-rd col.	4-th col.	5-th col.	6-th col.	7-th col.
ΔK^0	00000000	00000000	*****	00000000	00000000	*****	00000000	00000000
ΔS^1	00000000	00000000	00100000	00000000	00000000	00000100	00000000	00000000
ΔR^1	00100100	00000000	00000000	00000000	00000000	00000000	00000000	00000000
ΔK^1	00100100	00000000	00000000	00000000	00000000	00000000	00000000	00000000
ΔK^9	00010001	000*000*	00000000	00000000	00010001	000*000*	00000000	00000000
ΔS^{10}	00000000	*01**01*	00000000	00000000	00000000	*01**01*	00000000	00000000
ΔR^{10}	00000000	*000*000	000*000*	00100010	00000000	*000*000	000*000*	00100010
ΔK^{10}	00000000	*000*000	000*000*	00100010	00000000	*000*000	000*000*	00100010
ΔS^{11}	00000000	*****	*****	*****	00000000	*****	*****	*****
ΔR^{11}	0***0***	***0****	**0***0*	*0***0**	0***0***	***0***0	**0***0*	*0***0**
ΔK^{11}	0***0***	***0****	**0***0*	*0***0**	0***0***	***0***0	**0***0*	*0***0**

Table E18: Parameters and successive guessing and filtering steps for the 11-round FLY attack.

(a) Parameters		(b) Successive steps			
Symbol	Value	i	$g(i)$	$f(i)$	$\sum_{j=0}^i (g(j) - f(j))$
μ	4	0	8	0	8
p	56.77	1	8	8	8
N_{Δ, r_f}	2	2	8	8	8
M_{∇, r_b}	10	3	8	6	10
$\Lambda(\mu, p, d_{\text{in}})$	$4 \times 2^{56.77+1}$	4	8	6	12
d_{in}	16	5	8	6	14
d_{out}	48	6	8	6	16
k_{in}	16	7	8	0	24
k_{out}	76	8	6	0	30
c_e	$2^{6.46}$	9	6	4	32
K_m	128				
a	58				

The overall time complexity becomes:

$$\begin{aligned} \mathcal{T} \approx & 4 \times 2^{56.77+1+16-16} \cdot \{1 + 2^{-6.46} \cdot (2 + 2^8 + \dots + 2^{32})\} \\ & + 4 \times 2^{56.77+1} + 2^{128-58} = 2^{85.636} \quad (\text{encs}). \end{aligned}$$

Accordingly, the memory complexity is given by

$$\mathcal{M} = 8 \cdot 2^{16} + 8 \cdot 2^0 + 2^{16+76} \approx 2^{92} \quad (\text{bytes}).$$

In terms of data requirements, the attack needs

$$\mathcal{D} = 4 \times 2^{56.77+1} = 2^{59.77} \quad (\text{CP}).$$

Finally, the success probability of the attack is

$$\mathbb{P}_S = 0.9 \quad (90\%).$$

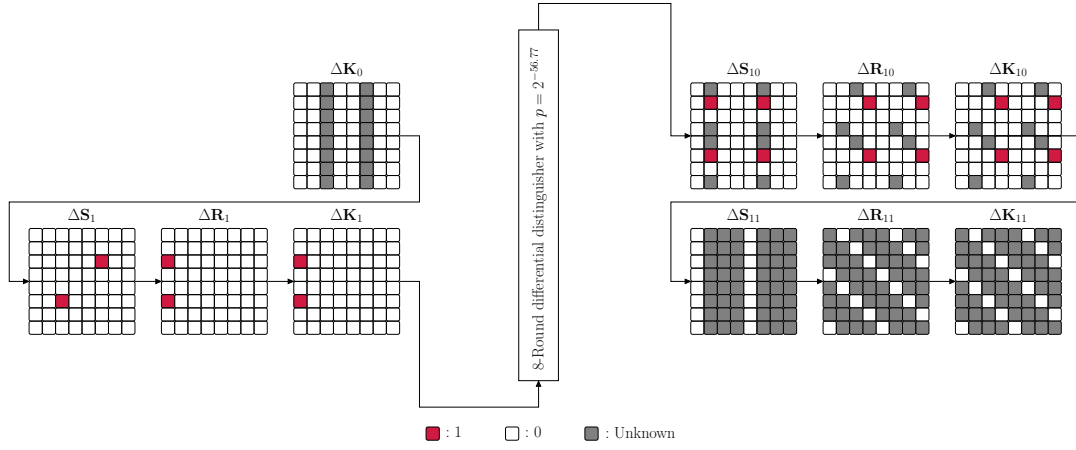


Fig. E2: 11-round key-recovery attack on FLY

Appendix F Details of the Key-Recovery Attack on 12-Round FLY

Table F19: Difference Values of ΔK^i , ΔS^i , and ΔR^i used in the key-recovery attack on 12-Round FLY.

	0-th col.	1-st col.	2-nd col.	3-rd col.	4-th col.	5-th col.	6-th col.	7-th col.
ΔK^0	*****	*****	*****	*****	*****	*****	*****	*****
ΔS^1	000*00*0	0000*00*	*0000*00	0*0000*0	00*0000*	*00*0000	0*00*000	00*00*00
ΔR^1	00000000	00000000	*****	00000000	00000000	*****	00000000	00000000
ΔK^1	00000000	00000000	*****	00000000	00000000	*****	00000000	00000000
ΔS^2	00000000	00000000	00100000	00000000	00000000	00000100	00000000	00000000
ΔR^2	00100100	00000000	00000000	00000000	00000000	00000000	00000000	00000000
ΔK^2	00100100	00000000	00000000	00000000	00000000	00000000	00000000	00000000
ΔK^{10}	00010001	000*000*	00000000	00000000	00010001	000*000*	00000000	00000000
ΔS^{11}	00000000	*01**01*	00000000	00000000	00000000	*01**01*	00000000	00000000
ΔR^{11}	00000000	*000*000	000*000*	00100010	00000000	*000*000	000*000*	00100010
ΔK^{11}	00000000	*000*000	000*000*	00100010	00000000	*000*000	000*000*	00100010
ΔS^{12}	00000000	*****	*****	*****	00000000	*****	*****	*****
ΔR^{12}	0***0***	***0***0	**0***0*	*0***0**	0***0***	***0***0	**0***0*	*0***0**
ΔK^{12}	0***0***	***0***0	**0***0*	*0***0**	0***0***	***0***0	**0***0*	*0***0**

References

- [1] Bogdanov, A., Knudsen, L.R., Leander, G., Paar, C., Poschmann, A., Robshaw, M.J., Seurin, Y., Vikkelsoe, C.: Present: An ultra-lightweight block cipher. In: Cryptographic Hardware and Embedded Systems-CHES 2007: 9th International Workshop, Vienna, Austria, September 10-13, 2007. Proceedings 9, pp. 450–466 (2007). Springer
- [2] Gérard, B., Grosso, V., Naya-Plasencia, M., Standaert, F.-X.: Block ciphers that are easier to mask: How far can we go? In: Cryptographic Hardware and Embedded Systems-CHES 2013: 15th International Workshop, Santa Barbara, CA, USA, August 20-23, 2013. Proceedings 15, pp. 383–399 (2013). Springer
- [3] Grosso, V., Leurent, G., Standaert, F.-X., Varicı, K.: Ls-designs: Bitslice encryption for efficient masked software implementations. In: Fast Software Encryption: 21st International Workshop, FSE 2014, London, UK, March 3-5, 2014. Revised Selected Papers 21, pp. 18–37 (2015). Springer
- [4] Zhang, W., Bao, Z., Lin, D., Rijmen, V., Yang, B., Verbauwhede, I.: Rectangle: a bit-slice lightweight block cipher suitable for multiple platforms. Cryptology ePrint Archive (2014)
- [5] Karpman, P., Grégoire, B.: The littlun s-box and the fly block cipher. In: Lightweight Cryptography Workshop (2016)
- [6] Kim, H., Jeon, Y., Kim, G., Kim, J., Sim, B.-Y., Han, D.-G., Seo, H., Kim, S., Hong, S., Sung, J., *et al.*: Pipo: A lightweight block cipher with efficient higher-order masking software implementations. In: Information Security and Cryptology-ICISC 2020: 23rd International Conference, Seoul, South Korea, December 2–4, 2020, Proceedings 23, pp. 99–122 (2021). Springer
- [7] Biham, E., Shamir, A.: Differential cryptanalysis of des-like cryptosystems. Journal of CRYPTOLOGY 4, 3–72 (1991)
- [8] Leander, G., Moos, T., Moradi, A., Rasoolzadeh, S.: The speedy family of block ciphers: Engineering an ultra low-latency cipher from gate level for secure processor architectures. IACR Transactions on Cryptographic Hardware and Embedded Systems, 510–545 (2021)
- [9] Boura, C., David, N., Heim Boissier, R., Naya-Plasencia, M.: Better steady than speedy: full break of speedy-7-192. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques, pp. 36–66 (2023). Springer
- [10] Zong, R., Dong, X., Chen, H., Luo, Y., Wang, S., Li, Z.: Towards key-recovery-attack friendly distinguishers: application to gift-128. IACR Transactions on Symmetric Cryptology, 156–184 (2021)

- [11] Hall-Andersen, M., Vejre, P.S.: Generating graphs packed with paths estimation of linear approximations and differentials. *IACR Transactions on Symmetric Cryptology*, 265–289 (2018)
- [12] Kim, S., Hong, D., Sung, J., Hong, S.: Accelerating the best trail search on aes-like ciphers. *IACR Transactions on Symmetric Cryptology*, 201–252 (2022)
- [13] Baek, S., Kim, G., Jeon, Y., Kim, J.: Enhancing the related-key security of pip through new key schedules. In: *International Conference on Information Security and Cryptology*, pp. 3–18 (2023). Springer
- [14] Lai, X., Massey, J.L., Murphy, S.: Markov ciphers and differential cryptanalysis. In: *Advances in Cryptology—EUROCRYPT’91: Workshop on the Theory and Application of Cryptographic Techniques Brighton, UK, April 8–11, 1991 Proceedings 10*, pp. 17–38 (1991). Springer
- [15] Selçuk, A.A.: On probability of success in linear and differential cryptanalysis. *Journal of Cryptology* **21**, 131–147 (2008)
- [16] Zhu, B., Dong, X., Yu, H.: Milp-based differential attack on round-reduced gift. In: *Topics in Cryptology—CT-RSA 2019: The Cryptographers’ Track at the RSA Conference 2019, San Francisco, CA, USA, March 4–8, 2019, Proceedings*, pp. 372–390 (2019). Springer
- [17] Zhang, Y., Sun, S., Cai, J., Hu, L.: Speeding up milp aided differential characteristic search with matsui’s strategy. In: *Information Security: 21st International Conference, ISC 2018, Guildford, UK, September 9–12, 2018, Proceedings 21*, pp. 101–115 (2018). Springer
- [18] Sun, L., Wang, W., Wang, M.: Accelerating the search of differential and linear characteristics with the sat method. *IACR Transactions on Symmetric Cryptology*, 269–315 (2021)
- [19] Wang, S., Feng, D., Hu, B., Guan, J., Zhang, K., Shi, T.: New method for combining matsui’s bounding conditions with sequential encoding method. *Designs, Codes and Cryptography* **91**(11), 3603–3642 (2023)
- [20] Boura, C., David, N., Derbez, P., Leander, G., Naya-Plasencia, M.: Differential meet-in-the-middle cryptanalysis. In: *Annual International Cryptology Conference*, pp. 240–272 (2023). Springer